A Mini Project Report on

FINE GRAINED TOPIC CATEGORIZATION OF BBC NEWS ARTICLES USING SUPERVISED DOCUMENT CLASSIFICATION MODELS

Submitted in partial fulfilment of the requirements
for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

By

K.SAI NIKETHAN (23UJ1A0584)
M.NAVEENA (23UJ1A0591)
M.ANKITHA (23UJ1A0587)
M.SAKSHI SREE (23UJ1A0592)

Under the esteemed guidance of

Dr.Zaheer Sultana M.Tech, (Ph.D)
Assistant Professor

Department of Computer Science and Engineering (Data Science)

at

MALLA REDDY ENGINEERING COLLEGE AND MANAGEMENT SCIENCES

Kistapur (V), Medchal (M), Medchal Malkajgiri (Dist)-501401.

(An UGC autonomous Institution, NBA Accredited in CSE, ECE, IT, EEE)

2023-2027

Affiliated to

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY HYDERABAD
KUKATPALLY, HYDERABAD-85.

MALLA REDDY ENGINEERING COLLEGE AND MANAGEMENT SCIENCES

Kistapur (V), Medchal (M), Medchal Malkajgiri (Dist)-501401.

(An UGC autonomous Institution, NBA Accredited in CSE, ECE, IT, EEE)

CERTIFICATE

This is to certify that the Mini project report entitled "**FINE GRAINED TOPIC CATEGORIZATION OF BBC NEWS ARTICLES USING SUPERVISED DOCUMENT CLASSIFICATION MODELS** "is the bonafide work carried out and submitted by

K.SAI NIKETHAN	**(23UJ1A0584)**
M.NAVEENA	**(23UJ1A0591)**
M.ANKITHA	**(23UJ1A0587)**
M.SAKSHI SREE	**(23UJ1A0592)**

To the department of **Computer Science and Engineering, Malla Reddy Engineering College and Management Sciences**, in partial fulfilment for the award of **BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING** during the academic year 2025-2026.

Internal Guide
Dr.Zaheer Sultana M.Tech,PhD.
Assistant Professor
Dept .of CSE (Data Science)

Head of The Department
Mr.G.Chakrapani M.Tech,(PhD).
Assistant Professor
Dept .of Computer Science and Engineering

EXTERNAL

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without the mention of people who made it possible, whose constant guidance and encouragement crowned our efforts with success. It is a pleasant aspect that I have now the opportunity to express my guidance for all of them.

We are thankful to **Mr.V.MALLA REDDY** chairman of Malla Reddy Engineering College and Management Sciences for accepting me and providing me with an opportunity to do a project in their esteemed organization.

We are thankful to Principal **Dr.M.SREEDHAR REDDY** M.Tech., Ph.D. Malla Reddy Engineering College and Management Sciences for helped me to undergo project work as a part of university curriculum.

Our special thanks to **Mr.G.CHAKRAPANI** M.Tech., (Ph.D). Assistant Professor& Head in Computer Science and Engineering Department and **Dr.ZAHEER SULTANA** M.Tech., (Ph.D). Assistant Professor in CSE (Data Science) Department for guiding me in the right way to complete my project in the right time.

We would like to thank our internal project mates and department faculties for their full-fledged guidance and giving courage to carry out the project.

We are very much thankful to one and all that helped us for the successful completion of our project.

`

K.SAI NIKETHAN (23UJ1A0584)
M.NAVEENA (23UJ1A0591)
M.ANKITHA (23UJ1A0587)
M.SAKSHI SREE (23UJ1A0592)

ABSTRACT

The rapid growth of digital journalism has resulted in an overwhelming volume of online news content, with global news platforms publishing millions of articles daily, making real-time topic identification and organization a critical challenge for information retrieval systems. In this context, fine-grained topic categorization has become essential for improving content recommendation, search efficiency, and user engagement. This work focuses on the fine-grained topic categorization of BBC news articles using supervised document classification models. Traditionally, news categorization has relied on manual systems where human editors read, analyze, and assign categories to articles. Although effective for small-scale datasets, manual classification is time-consuming, labor-intensive, inconsistent, and impractical for handling large and continuously growing news repositories. The limitations of the manual approach include high operational cost, scalability issues, delayed publication workflows, and the inability to maintain consistency across large datasets. These challenges motivate the need for an automated, accurate, and scalable text classification system. To address this problem, a machine learning–based framework is proposed that leverages Natural Language Processing (NLP) techniques and semantic feature extraction using GloVe word embeddings. The system performs text preprocessing, feature extraction, exploratory data analysis, and supervised learning to classify news articles into predefined topic labels. Existing models such as RadiusNeighborsClassifier, Decision Tree Classifier, and Gaussian Naïve Bayes are implemented as baseline approaches, while the K-Nearest Neighbors (KNN) classifier is proposed as the primary model for improved semantic similarity-based classification. Model performance is evaluated using accuracy, precision, recall, F1-score, confusion matrices, and AUC-ROC analysis. The experimental results demonstrate that the proposed system effectively automates the topic categorization process, reduces human dependency, improves classification accuracy, and provides a reliable solution for real-time news article organization, thereby addressing the core limitations of manual news categorization systems.

TABLE OF CONTENT

List of figures i
Abbreviations ii
Abstract iii

CHAPTER 1: INTRODUCTION

1.1 OVERVIEW 1
1.2 PROBLEM DEFINITION 1
1.3 RESEARCH MOTIVATION 2
1.4 RESEARCH OBJECTIVE 2
1.5 SIGNIFICANCE 2
1.6 APPLICATIONS 3
1.7 ADVANTAGES 3

CHAPTER 2: LITERATURE SURVEY 4

CHAPTER 3: EXISTING SYSTEM 18

CHAPTER 4: PROPOSED METHODODLOGY

4.1 OVERVIEW 20

4.2 PREPROCESSING 22

4.3 EXPLORATORY DATA ANALYSIS 24

4.4 GLOVE VECTOR 26

4.5 TRAIN TEST SPLIT 28

4.6 MODEL BUILDING AND TRAINING 29

4.6.1 RADIUS NEIGHBORS CLASSIFIER 30

4.6.2 DECISION TREE CLASSIFIER 32

4.6.3 GAUSSIAN NAÏVE BAYES 35

4.6.4 KNN CLASSIFIER 37

CHAPTER 5: UML DIAGRAMS 41

5.1 CLASS DIAGRAM 42

5.2 USE CASE DIAGRAM 43

5.3 DATA FLOW DIAGRAM 44

5.4 DEPLOYMENT DIAGRAM 44

5.5 COMPONENT DIAGRAM 45

5.6 COLLABORATION DIAGRAM 46

5.7 SEQUENCE DIAGRAM 47

CHAPTER 6: SOFTWARE ENVIRONMENT 49
6.1 SOFTWARE REQUIREMENTS 49
6.2 HARDWARE REQUIREMENTS 51

CHAPTER 7: FUNCTIONAL AND NON FUNCTIONAL REQUIREMENTS
7.1 FUNCTIONAL REQUIREMENTS 53
7.2 NON FUNCTIONAL REQUIREMENTS 54
7.3 SYSTEM STUDY 55

CHAPTER 8: SOURCE CODE 56

CHAPTER 9: RESULTS AND DISCUSSION 77
9.1 IMPLEMENTATION DESCRIPTION 77
9.2 DATASET DESCRIPTION 80
9.3 RESULTS DESCRIPTION 81
9.4 COMPARATIVE ANALYSIS 87

CHAPTER 10: CONCLUSION AND FUTURE SCOPE 89

REFERENCES 90

LIST OF FIGURES

Figure 3.1 Traditional System 18
Figure 4.1 Proposed System Architecture 20
Figure 4.2 Internal Working of NLP reprocessing 23
Figure 4.3 Internal working of EDA 25
Figure 4.4 Internal working flow of Glove vector 27
Figure 4.5 Internal working flow of Train test splitting 28
Figure 4.6 Internal working of Radius Neighbors classifier 30
Figure 4.7 Internal working of Decision tree classifier 33
Figure 4.8 Internal working of Gaussian Naïve bayes 36
Figure 4.9 Internal working of KNN classifier 38
Figure 5.1 Class Diagram 42
Figure 5.2 Use Case Diagram 43
Figure 5.3 Data flow diagram 44
Figure 5.4 Deployment Diagram 45
Figure 5.5 Component Diagram 46
Figure 5.6 Collaboration Diagram 47
Figure 5.7 Sequence Diagram 47
Figure 9.1 Sample records from the dataset 81
Figure 9.2 (a) Word Cloud of most frequent terms in dataset 82
Figure 9.2 (b) Top 20 most frequent words in the dataset 82
Figure 9.2 (c) Distribution of document lengths in e learning dataset 83
Figure 9.2 (d) Part-of-Speech Tag frequency Distribution 83
Figure 9.2 (e) Top 20 Most frequent Bigrams in the dataset 84
Figure 9.3 Confusion matrix analysis of classification models 85
Figure 9.4 Detailed Performance comparison of classification models 88

ABBREVATIONS

BBC – British Broadcasting Corporation

NLP – Natural Language Processing

ML – Machine Learning

KNN – K-Nearest Neighbors

EDA – Exploratory Data Analysis

TF-IDF – Term Frequency–Inverse Document Frequency

AUC – Area Under Curve

ROC – Receiver Operating Characteristic

AUC-ROC – Area Under the Receiver Operating Characteristic Curve

POS – Part of Speech

CSV – Comma-Separated Values

SVM – Support Vector Machine

CNN – Convolutional Neural Network

RNN – Recurrent Neural Network

OSN – Online Social Network

GSB – Graphical Set-Based Model

CART – Classification and Regression Tree

k-NN – k-Nearest Neighbor

NumPy – Numerical Python

CHAPTER 1
INTRODUCTION

1.1 Overview

Automatic text classification has been a salient research topic and had important applications ever since the rise in digital documents [1]. Text classification is now seen as a necessity due to the tremendous size of the text documents we see every day. A very significant application of text classification is topic based classification [2]. Intuitively, this is a task of classifying the text under a predetermined set of categories. If di is a set of D documents and there is a predetermined set of categories {c1,c2,c3...cn}, text classification then assigns one of the categories to cj to di [3]. Since a machine learning approach is used to perform classification, there is a requirement of an initial dataset that is used to perform training of the machine learning model [4]. The graphical representation of the Text Classification process One of the biggest issues in constructing a Machine learning classifier is the number of features that can easily reach very high numbers of tens of thousands [5], in turn making it extremely slow to compute the values of the labels [6]. To address this problem, there is a need of dimension reduction methods to be implemented. One of which is by selecting the subset of the original feature [7]. The other is by transforming the older features into new ones [8]. After which machine learning algorithms can be applied and then we can determine which algorithm produces better results [9]. Typically, Support Vector Machines produce better accuracy and are more widely used for text classification. However, there are various other algorithms that can be used. A few of them being the Naive Bayes classifier, Random Forest classifier, and Logistic Regression which we implemented to compare the results [10].

1.2 Problem Definition

The exponential growth of digital news content has made it increasingly difficult to efficiently organize, manage, and retrieve relevant information in real time. News platforms such as BBC generate a massive volume of articles across multiple domains including business, politics, sports, technology, and entertainment. Traditional keyword-based or manual classification methods fail to capture semantic context and are not scalable for large datasets. This results in delayed categorization, inconsistent labeling, and reduced effectiveness of search and recommendation systems. Therefore, there is a critical need for an

automated, accurate, and scalable fine-grained topic categorization system that can classify news articles based on semantic understanding using supervised machine learning techniques.

1.3 Research Motivation

The motivation for this research arises from the limitations of manual and rule-based news classification systems, which are labor-intensive, error-prone, and incapable of handling continuously growing textual data. With advancements in Natural Language Processing and word embedding techniques such as GloVe, it has become possible to capture semantic relationships between words and documents. Additionally, supervised machine learning models like RadiusNeighborsClassifier, Decision Tree Classifier, Gaussian Naïve Bayes, and K-Nearest Neighbors offer promising solutions for efficient document classification. This research is motivated by the need to evaluate these models and develop a reliable automated system that enhances classification accuracy while reducing human effort and operational cost.

1.4 Research Objectives

- To preprocess and clean BBC news articles using NLP techniques.
- To extract semantic features from text using GloVe word embeddings.
- To perform exploratory data analysis to understand textual patterns and label distributions.

1.5 Significance

- Enables automated classification of large-scale BBC news articles, reducing reliance on manual labeling.
- Improves semantic understanding of textual data through the use of GloVe word embeddings.
- Enhances the accuracy and consistency of topic categorization across multiple news domains.
- Demonstrates the effectiveness of supervised machine learning models such as RadiusNeighborsClassifier, Decision Tree Classifier, Gaussian Naïve Bayes, and K-Nearest Neighbors.

- Supports fine-grained topic-level classification rather than coarse or keyword-based grouping.
- Facilitates efficient indexing, retrieval, and organization of news articles.
- Provides a scalable solution suitable for real-time and continuously growing news datasets.
- Assists researchers and developers in understanding comparative model performance using standard evaluation metrics.
- Serves as a foundation for future advancements in NLP-based document classification systems.
- Contributes to improved content recommendation and user experience in digital news platforms.

1.6 Applications

- Automated news categorization for digital journalism platforms.
- Content recommendation systems in news and media applications.
- Efficient indexing and retrieval in search engines.
- Real-time monitoring and analysis of news trends.
- News aggregation and topic-based content filtering systems.
- Support systems for journalists and editors to manage large news repositories.

1.7 Advantages

- Fully automated topic classification with minimal human intervention.
- Scalable and suitable for large and continuously growing datasets.
- Semantic understanding of text using GloVe embeddings.
- Comparative evaluation of multiple supervised learning algorithms.
- Supports real-time, single-text, and batch-wise predictions.
- Improves accuracy, consistency, and efficiency over manual classification systems.
- Cost-effective and time-efficient solution for news content management.

CHAPTER 2

LITERATURE SURVEY

Vidulin et al. [11] proposed Automatic text classification is a crucial method to manage and process vast amounts of textual data in the digital form. They illustrates the text classification and text summarization using machine learning techniques and algorithms. An important part of this task is that the input data is in the form of natural language so we must do some preliminary analysis to transform the documents into a uniform structure so that we can train our data using numerical representations.

Daud et al. [12] explored Much news is available online, and not all is categorized. A few researchers have carried out work on news classification in the past, and most of the work focused on fake news identification. Most of the work performed on news categorization is carried out on a benchmark dataset. The problem with the benchmark dataset is that model trained with it is not applicable in the real world as the data are pre-organized. This study used machine learning (ML) techniques to categorize online news articles as these techniques are cheaper in terms of computational needs and are less complex.

Rafat et al. [13] presented substantial amount of data is generated day-to-day, to which news articles are a major contributor. Most of this data is not well-structured, highlighting the need for efficient ways to manage, process, and analyze said data. One useful approach involves the categorization of the data.

Piasta et al. [14] considered aspects such as hardware resource management, implementation simplicity, learning time, and the quality of the resulting model in terms of detection, and it examined a range of techniques for attribute selection, feature filtering, vector representation, and the handling of imbalanced datasets. Advanced techniques for word selection and named entity recognition were employed.

Masri et al. [15] obtained actionable information for humanitarian and other emergency responses, an accurate classification of news or events is critical. Daily news and social media are hard to classify based on conveyed information, especially when multiple categories of information are embedded.

Kalogeropoulos et al. [16] aimed to enhance the Graphical Set-based model (GSB) for ranking and classification tasks by incorporating node and word embeddings. The model integrates a textual graph representation with a set-based model for information retrieval. Initially, each document in a collection is transformed into a graph representation. The

proposed enhancement involves augmenting the edges of these graphs with embeddings, which can be pretrained or generated using Machine learning models.

Babić et al. [17] proposed natural language processing, text needs to be transformed into a machine-readable representation before any processing. The quality of further natural language processing tasks greatly depends on the quality of those representations. In this survey, we systematize and analyze 50 neural models from the last decade. The models described are grouped by the architecture of neural networks as shallow, recurrent, recursive, convolutional, and attention models.

Harris et al. [18] obtained emergence and acceptance of digital technology have caused information pollution and an infodemic on Online Social Networks (OSNs), blogs, and online websites. The malicious broadcast of illegal, objectionable and misleading content causes behavioural changes and social unrest, impacts economic growth and national security, and threatens users' safety.

Petukhova et al. [19] presented a dataset of 10,917 news articles with hierarchical news categories collected between 1 January 2019 and 31 December 2019. We manually labeled the articles based on a hierarchical taxonomy with 17 first-level and 109 second-level categories. This dataset can be used to train machine learning models for automatically classifying news articles by topic. This dataset can be helpful for researchers working on news structuring, classification, and predicting future events based on released news.

Yuan et al. [20] presented With the popularization of digital technology, the problem of information pollution caused by fake news has become more common. Malicious dissemination of harmful, offensive or illegal content may lead to misleading, misunderstanding and social unrest, affecting social stability and sustainable economic development. With the continuous iteration of artificial intelligence technology, researchers have carried out automatic and intelligent news data mining and analysis based on aspects of information characteristics and realized the effective identification of fake news information.

Misra, et al. [21] introduced the News Category Dataset, which played a key role in advancing research on news article classification. The dataset includes a wide range of labeled news articles grouped into specific topic categories rather than general ones. This allowed researchers to test how well classification models perform when topics are closely related. The structure of the dataset closely matches the writing style of BBC news, making it realistic for experimentation. It also helped establish a common benchmark for comparing different classification methods. As a result, many later studies relied on this dataset for fine-grained topic analysis.

D212Digital, et al. [22] provided a comprehensive explanation of n-gram-based feature engineering for text processing tasks. Their work clearly explained how breaking text into sequences of words helps capture contextual meaning. The study showed that single words alone may not always represent topic intent accurately. By using combinations of words, classification systems can better understand phrases commonly used in news writing. This approach is especially helpful in distinguishing similar topics. Their guide served as a practical reference for improving text representation.

Manoj, et al. [23] proposed a multi-class Support Vector Machine model for classifying newspaper articles. Instead of handling categories separately, their approach classified all topics together in a single framework. This reduced overlap and confusion between similar news categories. The study showed improved accuracy compared to binary classification approaches. Their results demonstrated that real-world news classification benefits from multiclass learning. This work highlighted the importance of treating news topics as interconnected rather than isolated.

Nigam, et al. [24] conducted a detailed comparison of popular classification algorithms used in text categorization. The study evaluated Naïve Bayes, Support Vector Machines, and Logistic Regression on news datasets. Their results showed that algorithm performance varies depending on topic similarity and data distribution. The authors emphasized that closely related news topics require more robust models. This study helped researchers understand the strengths and weaknesses of different classifiers. It also guided model selection for fine-grained classification tasks.

Foroughi, et al. [25] developed a supervised multi-class text classification framework focused on news-style datasets. Their research highlighted the role of data preprocessing in improving classification accuracy. The study demonstrated that cleaning, normalization, and feature extraction directly influence model performance. They showed that even strong algorithms fail when text quality is poor. Their framework achieved better results by combining proper preprocessing with supervised learning. This work reinforced the importance of end-to-end system design.

Duan, et al. [26] presented a complete supervised news classification system using real-world data. Their work focused on building a structured pipeline from data collection to classification. The study showed that organizing each stage improves consistency and reliability. Their system successfully separated news articles into detailed topic categories. The authors demonstrated that practical implementations require careful workflow planning. This research contributed to applied news classification systems.

Barbaresi, et al. [27] introduced Trafilatura, a tool designed to extract clean text from news websites. Their work addressed the challenge of removing irrelevant content such as ads and navigation menus. The study showed that accurate text extraction improves classification outcomes. Clean input data leads to more reliable feature representation. Their tool helped researchers obtain high-quality datasets from online news sources. This work emphasized that data extraction is a critical first step.

Germec, et al. [28] analyzed the impact of text preprocessing techniques on classification performance. Their study examined normalization, punctuation removal, and text cleaning. Results showed that improper preprocessing can significantly reduce accuracy. The authors highlighted that news articles contain noise that must be handled carefully. Their findings emphasized the need for consistent preprocessing strategies. This research helped standardize text preparation practices.

Silva, et al. [29] outlined essential preprocessing steps for natural language tasks. Their work demonstrated how tokenization and noise removal improve feature clarity. The study showed that structured preprocessing leads to better learning outcomes. News articles often contain formatting issues that affect classification. Their findings helped improve text quality before model training. This work contributed to improved preprocessing pipelines.

Sarica, et al. [30] examined how stopwords affect classification accuracy. The study showed that blindly removing stopwords is not always beneficial. In news text, some common words contribute to meaning and context. Their research highlighted the need for dataset-specific stopword handling. The authors showed that thoughtful preprocessing improves classification reliability. This work informed better feature design choices.

Praveen Gujjar, et al. [31] proposed a scalable framework for large-scale text processing. Their system handled diverse datasets, including news articles. The study showed improved classification accuracy through efficient processing techniques. Their approach addressed challenges related to data volume and complexity. This work demonstrated the need for scalable solutions in modern news platforms. It supported real-world deployment scenarios.

Aditya, et al. [32] compared Naïve Bayes and Maximum Entropy models for classification tasks. Their study showed that Maximum Entropy models better capture contextual dependencies. This advantage is important when topics share overlapping vocabulary. The authors highlighted how context improves classification accuracy. Their results supported the use of more expressive models. This study guided classifier selection for fine-grained tasks.

Mazuelas, et al. [33] extended Maximum Entropy models to handle complex feature spaces. Their research showed that these models remain stable with large vocabularies. This is

particularly useful for news datasets with diverse terminology. The study demonstrated robustness against overfitting. Their work improved reliability in supervised classification. It contributed to handling high-dimensional text data.

Kruczek, et al. [34] studied the role of n-gram features in classification. Their results showed that combining single words and word pairs improves topic distinction. The approach captured meaningful expressions used in news articles. Their study showed improved discrimination between similar categories. This work guided feature engineering strategies. It reinforced the importance of phrase-level features.

Das, et al. [35] analyzed TF-IDF weighting methods for unstructured text. Their study showed that TF-IDF highlights important topic-specific words. It reduces the influence of frequently occurring but less meaningful terms. This improved class separation in news datasets. Their findings supported TF-IDF as a reliable weighting scheme. This work strengthened feature representation techniques.

Brownlee, et al. [36] explained the Bag-of-Words model in detail. Despite its simplicity, the model remained effective for classification tasks. The study showed that it serves as a strong baseline. Many advanced methods still build upon it. Their explanation helped researchers understand foundational concepts. This work remains relevant in text classification research.

Gupta, et al. [37] reviewed feature selection techniques used in machine learning. Their work showed that removing irrelevant features improves accuracy. The study emphasized reducing complexity without losing information. This is important for large news datasets. Their findings supported efficient model training. This research guided feature optimization.

Morán-Fernández, et al. [38] studied feature reduction strategies for classification tasks. Their results showed improved performance on large text datasets. The study highlighted the importance of selecting meaningful features. Reduced dimensionality improved training efficiency. Their work supported scalable classification systems. It was particularly relevant for news analysis. Bayat, et al. [39] evaluated advanced neural architectures for text processing. Their findings showed improved semantic understanding. The study demonstrated better pattern recognition in news articles. These models captured deeper meaning beyond keywords. Their work supported advanced approaches for classification. It contributed to semantic-level analysis. Merritt, et al. [40] explained Transformer models and their benefits. Their work showed how attention mechanisms improve context understanding. The study highlighted suitability for long documents. News articles often require full-document analysis. Their explanation supported adoption in classification tasks. This work clarified modern modeling techniques. Bengi, et al. [41] presented an introductory

explanation of Transformer architectures and their role in language-related tasks. Their work focused on explaining how attention mechanisms allow models to focus on important parts of text. This ability helps in understanding relationships between words across sentences. The authors showed that Transformers perform better than traditional models when handling complex language patterns. Their discussion made advanced concepts easier to understand for beginners. This work encouraged wider adoption of Transformer-based models in news classification.

Khay, et al. [42] analyzed popular Transformer-based models and compared their performance across different language processing tasks. Their study highlighted the strengths of attentionbased architectures in capturing context. The authors showed that these models perform consistently well on classification tasks involving large documents. Their analysis confirmed that Transformers are suitable for news articles with detailed content. The work provided insights into choosing appropriate models for text classification. It supported the shift toward modern deep learning approaches.

Rao V and Sachdev J, et al. [43] proposed a machine learning approach for classifying news articles based on geographical location. Their work focused on identifying location-specific patterns within news content. The study demonstrated that textual cues can effectively indicate regional relevance. Their model achieved good accuracy using supervised learning techniques. This research showed the importance of contextual features in news classification. It also highlighted applications in regional news filtering and analysis.

V, M., T, D., and Kalaiyarasi, et al. [44] compared Support Vector Machine classifiers with Perceptron models for newspaper article classification. Their study showed that SVMs performed better in terms of accuracy and stability. The authors explained that SVMs handle high-dimensional text data more effectively. Their experiments demonstrated improved classification of complex news categories. This work supported the preference for SVMs in text-based applications. It provided practical evidence for algorithm selection.

Jia, et al. [45] studied reader emotion classification using news headlines. Their research focused on understanding how headlines influence emotional responses. The study showed that short text fragments still carry strong emotional signals. Their classification approach successfully identified emotions such as anger, joy, and sadness. This work highlighted the role of sentiment in news analysis. It also showed how classification techniques extend beyond topic detection.

Beltagy, et al. [46] introduced the Longformer model to address challenges associated with long documents. Their approach used a sparse attention mechanism to reduce computational

cost. This allowed the model to process long news articles without losing important context. The study showed improved performance on document-level tasks. Their work addressed limitations of standard Transformer models. It was particularly useful for full-length news classification.

Vaswani, et al. [47] introduced the Transformer architecture based entirely on attention mechanisms. Their work removed the need for recurrence and convolution. This allowed faster training and better context handling. The model showed strong performance across language tasks. Their architecture became the foundation for many modern classification systems. This work marked a major shift in text processing research.

Ott, et al. [48] investigated large-scale training strategies for neural models. Their study focused on improving efficiency and accuracy during training. The authors showed that careful optimization improves classification results. Their work addressed challenges related to scalability. This research helped improve deep learning model performance. It contributed to practical deployment of large models.

Popel, et al. [49] provided guidelines for training Transformer models effectively. Their work emphasized proper parameter tuning and training stability. The study showed that following best practices improves convergence. This leads to better classification accuracy. Their guidelines were useful for researchers working with large datasets. This work helped reduce training-related errors.

Keskar, et al. [50] analyzed the impact of large-batch training on model generalization. Their study showed that very large batches can reduce performance. The authors explained the tradeoff between training speed and accuracy. Their findings helped improve training strategies. This research guided better model optimization. It contributed to stable training of classification models.

Rish, et al. [51] conducted an empirical study on the Naïve Bayes classifier. Their work showed that it remains effective despite its simplicity. The classifier performed well on text categorization tasks. Their results highlighted its efficiency and reliability. This work reinforced Naïve Bayes as a strong baseline. It remains widely used in news classification.

Ray, et al. [52] explained real-world applications of Naïve Bayes classifiers. Their study demonstrated ease of implementation and fast training. The authors showed that it performs well on large text datasets. This made it suitable for news categorization. Their work emphasized practical usefulness. It supported continued use of probabilistic models.

Berger, et al. [53] introduced Maximum Entropy models for natural language tasks. Their approach allowed flexible integration of diverse features. This improved handling of complex

language patterns. The study showed strong performance in classification tasks. Their work influenced later text classification research. It provided a strong alternative to probabilistic models.

Maroulis, et al. [54] compared Maximum Entropy and Naïve Bayes classifiers. Their study highlighted differences in accuracy and complexity. The authors explained trade-offs between simplicity and expressiveness. Their findings helped in choosing suitable models. This comparison was useful for applied text classification. It guided decision-making in model selection.

Ganitkevitch, et al. [55] introduced the Paraphrase Database, which provides a large collection of phrases and sentences that share similar meanings. This resource was designed to handle the problem of language variation, where the same idea can be expressed in many different ways. In news articles, different journalists often use different wording to describe the same event. The database helped classification systems recognize such variations without treating them as separate topics. By improving semantic understanding, it reduced misclassification caused by vocabulary differences. This work significantly improved robustness in supervised text classification tasks.

Manning, et al. [56] presented fundamental concepts in information retrieval and text classification that continue to influence modern research. Their work explained core ideas such as document representation, term weighting, similarity measures, and classification strategies. These principles form the theoretical backbone of many supervised news classification systems. The authors emphasized the importance of understanding both statistical and linguistic aspects of text. Their explanations helped bridge the gap between theory and practical application. This work remains a standard reference for students and researchers working on text-based systems.

Karpiuk, et al. [57] investigated sequence pattern-based approaches for document classification. Their research focused on capturing the order of words rather than treating text as an unordered collection. This approach helped identify meaningful patterns that appear consistently in specific topics. In news articles, sentence structure and phrase sequence often indicate the topic more clearly. Their study showed that considering sequence information improves classification accuracy. This work contributed to more structured and context-aware text classification methods.

Mohammed Badr, et al. [58] explored the use of skip-grams and part-of-speech features to enhance text classification. Skip-grams allowed the model to capture relationships between words that are not directly adjacent. Part-of-speech information helped distinguish how words

function within sentences. Together, these features provided richer representations of text content. Their study showed improved performance in distinguishing closely related topics. This work demonstrated that combining syntactic and semantic features strengthens finegrained classification.

Robinson, et al. [59] analyzed the impact of bigram and trigram language models on classification performance. Their work showed that phrase-level features capture meaningful expressions that single words may miss. In news writing, repeated phrases often indicate specific topics or events. The study demonstrated that higher-order n-grams improve topic separation when used carefully. However, they also highlighted the need to manage increased feature size. Their findings guided balanced feature selection in news classification systems.

Berger, et al. [60] made early contributions to probabilistic language modeling, which influenced later developments in text classification. Their work explained how probability distributions can represent language patterns effectively. These models helped estimate the likelihood of words appearing in specific topics. This approach allowed systems to handle uncertainty and ambiguity in text. Their ideas formed the foundation for many supervised and unsupervised classification techniques. This contribution played a key role in shaping modern document classification research.

Pouli, V. et al. [61] presented a personalized multimedia retrieval system based on relevance feedback mechanisms. The authors focus on improving user experience by incorporating both explicit and implicit feedback into the retrieval loop. Their framework dynamically updates similarity measures according to user preferences. Machine learning techniques are applied to refine ranking results over multiple iterations. Experimental validation shows improved retrieval precision and user satisfaction. The system effectively adapts to changing user interests over time. The study emphasizes the importance of personalization in intelligent multimedia information systems.

Thai, M.T. et al. [62] presented analytical techniques for handling big data in complex and social network structures. It discusses graph theory foundations, network modeling, and scalable mining algorithms. Topics such as community detection, influence propagation, and anomaly detection are covered in detail. The authors address computational challenges in processing large-scale network datasets. Practical applications include social media analytics, cybersecurity, and biological networks. Emphasis is placed on parallel and distributed computing solutions. The book serves as a comprehensive reference for big data-driven network research.

Li, H. & Yamanishi, K. [63] proposed an ESC-based stochastic decision list model for text classification. The method integrates extended stochastic complexity with probabilistic decision lists. It aims to balance model accuracy and interpretability. The approach effectively handles uncertainty and sparse textual features. Comparative experiments demonstrate competitive performance against traditional classifiers. The model reduces overfitting by optimizing complexity measures. This research contributes to probabilistic modeling techniques in text categorization.

Kadhim, A.I. [64] comprehensively reviewed supervised machine learning algorithms for text classification. It analyzes traditional models such as Naïve Bayes, k-NN, Decision Trees, and Support Vector Machines. The paper also discusses deep learning approaches including CNNs and RNNs. Feature extraction techniques like TF-IDF and n-grams are examined. Evaluation metrics such as accuracy, precision, recall, and F1-score are compared. Advantages and limitations of each classifier are clearly summarized. The survey identifies open challenges in multilingual and large-scale text analytics.

Ko, Y. & Seo, J. [65] introduced an unsupervised approach to automatic text categorization. The authors reduce reliance on labeled training datasets by using clustering-based techniques. Documents are grouped according to similarity metrics and feature distributions. The approach automatically generates category labels from dominant terms. Experimental results show promising classification performance without supervision. The study addresses scalability for large document collections. It laid foundational work for later research in unsupervised NLP methods.

Kowsari, K. et al. [66] provided an extensive overview of classical and modern text classification algorithms. It categorizes techniques into supervised, unsupervised, and deep learning-based methods. The authors discuss feature representation strategies such as bag-of-words and word embeddings. Performance comparisons are made across benchmark datasets. The survey evaluates scalability and computational complexity issues. It highlights emerging trends such as transfer learning and neural architectures. The paper serves as a roadmap for future research in text mining.

Pedregosa, F. et al. [67] introduced Scikit-learn, an open-source Python library for machine learning. It provides efficient implementations of classification, regression, clustering, and preprocessing tools. The framework is built on NumPy and SciPy for optimized performance. It emphasizes modularity, ease of use, and consistent API design. The library supports model evaluation and cross-validation techniques. Extensive documentation and community support

enhance usability. Scikit-learn has become a standard tool in academic and industrial ML research.

Maas, A.L. et al. [68] proposed distributed word representations for sentiment classification. The authors train word vectors using a large movie review dataset. The learned embeddings capture semantic and syntactic relationships between words. The approach improves sentiment prediction compared to traditional bag-of-words models. It demonstrates better generalization across domains. Experimental evaluation shows significant accuracy improvements. The work influenced later neural embedding techniques in NLP.

Pradhan, L. et al. [69] compared different machine learning classifiers on news article datasets. Algorithms such as Naïve Bayes, SVM, and Decision Trees are evaluated. Feature selection and preprocessing methods are considered. Performance is measured using accuracy, precision, and recall metrics. Results indicate SVM provides superior performance in high-dimensional data. The study highlights trade-offs between computational cost and accuracy. It offers practical guidance for classifier selection in news analytics.

Rennie, J.D.M. & Rifkin, R. [70] addressed challenges in extending SVM to multiclass classification. They compare one-vs-rest and other decomposition strategies. Feature normalization techniques are proposed to enhance stability. Experimental results show improved accuracy on text datasets. The paper analyzes computational efficiency issues. It demonstrates SVM's scalability to multiple categories. This work significantly advanced multiclass text classification research.

Ghosh, M. & Sanyal, G. [71] investigated about ensemble-based feature selection for sentiment analysis. Multiple classifiers are integrated to enhance predictive performance. The approach reduces redundancy and noise in textual features. Comparative experiments show improved robustness. The ensemble framework enhances classification stability. The study demonstrates better generalization capability. It emphasizes hybrid techniques in opinion mining systems.

Gamal, D. et al. [72] evaluated machine learning algorithms for cross-domain opinion mining. SVM, Naïve Bayes, and Decision Trees are compared. Domain-specific challenges such as vocabulary variation are analyzed. Performance differences across datasets are discussed. The study highlights feature engineering importance. Results show no single model dominates all domains. It underscores adaptability in sentiment analysis applications.

Joachims, T. [73] pioneered work establishes Support Vector Machines (SVM) as a highly effective method for text categorization. The study focuses on handling extremely high-dimensional and sparse feature spaces that naturally arise from bag-of-words text

representations. By applying the principle of margin maximization, SVM achieves strong generalization performance even when the number of features exceeds the number of training samples. The research provides a detailed comparison with traditional classifiers such as k-Nearest Neighbor and Naïve Bayes. Experimental results demonstrate that SVM consistently achieves higher accuracy and robustness across benchmark datasets. The paper also analyzes computational feasibility and scalability for large document collections. This work becomes foundational in statistical learning theory applications to information retrieval and significantly influences later research in text mining and NLP.

Do, T.N. et al. [74] introduced an advanced Random Forest model incorporating oblique decision trees instead of conventional axis-parallel splits. Unlike traditional trees that split using a single feature, oblique trees create decision boundaries using linear combinations of multiple features, enabling more flexible partitioning of complex data. The approach effectively addresses challenges associated with very high-dimensional datasets, including feature redundancy and correlated attributes. Extensive experiments compare the proposed model with classical Random Forest and standard decision tree methods. Results indicate improved classification accuracy and better boundary approximation in complex classification tasks. The authors also examine computational overhead and optimization strategies to maintain scalability. The study significantly advances ensemble learning techniques for high-dimensional structured and textual data.

Aggarwal, C.C. [75] presented a detailed exploration of machine learning methodologies specifically designed for text data. It systematically covers preprocessing steps such as tokenization, stop-word removal, stemming, and vector space modeling. The book discusses classical algorithms including Naïve Bayes, SVM, and Decision Trees, alongside modern deep learning architectures like CNNs, RNNs, and attention-based models. Advanced topics such as topic modeling, sentiment analysis, document clustering, and information extraction are thoroughly explained. Mathematical foundations underlying probabilistic and optimization-based models are clearly derived. The author also addresses scalability issues in large-scale text mining and streaming data environments. With theoretical depth and practical case studies, the book serves as a definitive reference for both academic researchers and industry practitioners in text analytics.

Salazar, D.A. et al. [76] provided a systematic comparison between Support Vector Machines and Logistic Regression for binary classification problems. It evaluates model performance using statistical discrimination metrics such as ROC curves, sensitivity, specificity, and overall accuracy. The research demonstrates that SVM performs particularly well in

nonlinear and high-dimensional feature spaces. In contrast, Logistic Regression offers interpretable probabilistic outputs and straightforward parameter estimation. The authors analyze underlying assumptions such as linear separability and independence of predictors. Computational efficiency, convergence properties, and regularization effects are also examined. The study provides practical insights into model selection criteria based on dataset structure and application requirements.

Sperandei, S. [77] provided a detailed conceptual explanation of logistic regression analysis with emphasis on interpretation and practical application. It explains how regression coefficients relate to odds ratios and probability estimation. The study clarifies assumptions including independence of observations, absence of multicollinearity, and linearity in the logit function. Goodness-of-fit measures such as likelihood ratio tests and pseudo R-squared values are discussed. Real-world biomedical examples demonstrate how logistic regression models binary outcome variables. The paper highlights common interpretation errors and misuses of regression outputs. It serves as an essential instructional resource for researchers applying statistical classification methods.

Yue, S. et al. [78] presented a theoretical overview of Support Vector Machine classification principles. It explains the mathematical formulation of SVM as a convex optimization problem aimed at maximizing the separating margin. Kernel functions such as linear, polynomial, and radial basis functions are examined for nonlinear decision boundaries. The authors discuss challenges in parameter selection, particularly tuning the regularization parameter and kernel hyperparameters. Scalability concerns for large datasets and computational complexity of quadratic programming are analyzed. The study also highlights limitations such as sensitivity to noise and imbalance in training data. It provides direction for improving algorithmic efficiency and practical implementation of SVM models.

Kotsiantis, S.B. [79] provided an in-depth review of decision tree learning algorithms and their recent enhancements. It compares major algorithms such as ID3, C4.5, CART, and hybrid tree-based approaches. The study explains splitting criteria including information gain, gain ratio, and Gini impurity measures. Pruning strategies such as pre-pruning and post-pruning are analyzed to reduce overfitting. Advantages such as model interpretability, simplicity, and ease of implementation are emphasized. Limitations including instability due to small data variations and bias toward dominant attributes are critically examined. The survey offers a balanced and analytical evaluation of decision tree methodologies across various application domains.

Kamiński, B. et al. [80] proposed a structured framework for conducting sensitivity analysis in decision tree models. It examines how variations in input parameters influence classification outcomes and decision paths. The approach enhances interpretability by identifying critical variables that significantly impact predictions. Robustness analysis is performed to assess model stability under uncertain or noisy data conditions. Case studies demonstrate practical applications in operational research and decision-support systems. The framework integrates risk assessment and parameter uncertainty evaluation into tree-based modeling. This research strengthens confidence in predictive analytics by improving transparency and reliability of decision tree classifiers.

CHAPTER 3

EXISTING SYSTEM

In traditional news management systems, topic categorization of news articles is predominantly performed using manual or semi-automated approaches. Human editors or content managers read each news article and assign it to predefined categories such as business, politics, sports, technology, or entertainment based on their understanding of the content. In some cases, basic keyword-based or rule-based filtering systems are used to assist manual classification. These systems rely heavily on fixed vocabularies, handcrafted rules, and human judgment. While such approaches may work effectively for small-scale datasets, they become inefficient when applied to large and continuously growing news repositories like BBC news archives.

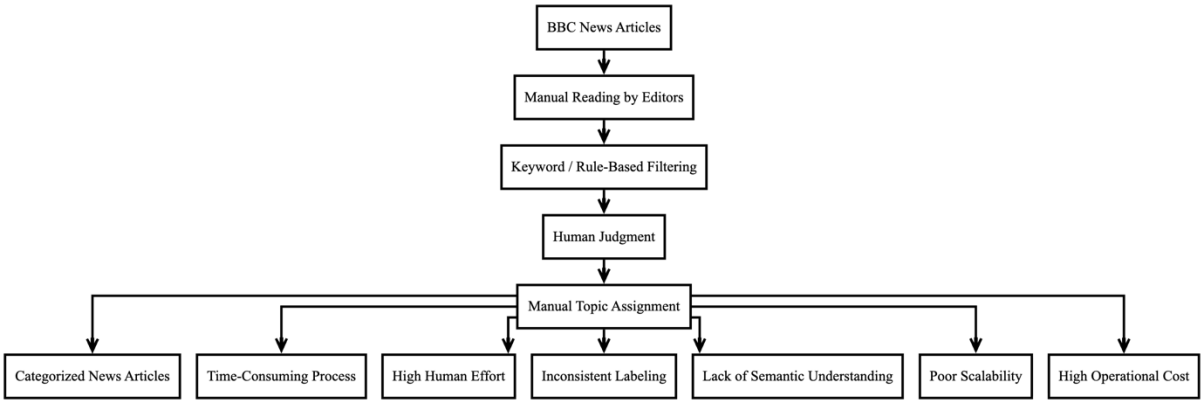


Fig. 3.1: Traditional System.

The traditional system lacks semantic understanding of text, making it difficult to handle linguistic variations, contextual meanings, and fine-grained topic distinctions. As a result, these methods struggle to meet the real-time demands and accuracy requirements of modern digital journalism platforms. In addition to manual classification, traditional news categorization systems often depend on simple rule-based or keyword-matching techniques, where predefined vocabularies and fixed patterns are used to assign categories to news articles. These systems lack adaptability and fail to evolve with changing language usage, emerging topics, and new terminologies frequently observed in news media. The absence of advanced Natural Language Processing techniques prevents traditional systems from understanding contextual meaning, semantic similarity, and sentence structure, leading to

inaccurate or overly generic classifications. Furthermore, such systems are unable to efficiently process multilingual content or handle ambiguous articles that span multiple topics. As news platforms increasingly demand real-time categorization for content recommendation and personalized delivery, traditional systems fall short in meeting performance, accuracy, and scalability requirements. Consequently, these limitations highlight the necessity for intelligent, automated, and data-driven classification approaches that can effectively manage large-scale news datasets while maintaining consistency and reliability.

Limitations

- Highly time-consuming due to manual reading and categorization of articles
- Requires significant human effort and domain expertise
- Prone to inconsistency and subjective bias in topic labeling
- Not scalable for large and continuously updating news datasets
- Delays in publishing and content organization workflows
- Ineffective in capturing semantic relationships and contextual meaning
- Keyword-based systems fail to handle synonyms and language variations
- High operational and maintenance cost
- Limited adaptability to new or emerging news topics

CHAPTER 4

PROPOSED METHODOLOGY

4.1 Overview

The proposed methodology presents a systematic and data-driven approach for fine-grained topic categorization of BBC news articles using supervised document classification models. The system is designed to automatically process large volumes of unstructured textual news data and accurately classify articles into predefined topic categories. The methodology follows a structured pipeline that begins with dataset ingestion and NLP-based text preprocessing, followed by semantic feature extraction using GloVe word embeddings. Exploratory Data Analysis (EDA) is conducted to understand linguistic patterns and dataset characteristics. Multiple supervised machine learning models are trained and evaluated, including existing baseline classifiers and a proposed K-Nearest Neighbors model. Model performance is analyzed using standard evaluation metrics and visualization techniques. The trained system supports both single-text and batch-wise predictions, enabling real-time and scalable news categorization. This automated framework eliminates the limitations of manual classification and ensures consistent, efficient, and accurate topic identification.

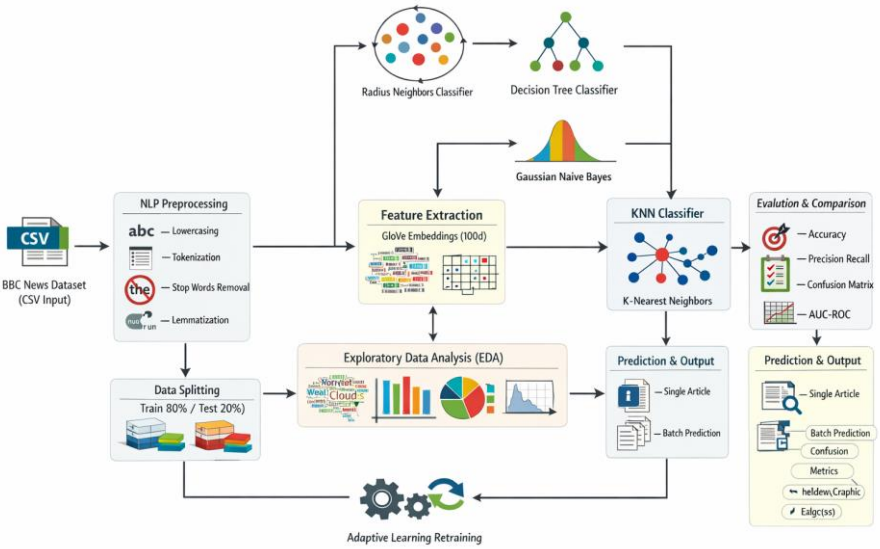


Fig. 4.1: Proposed System Architecture.

Data Ingestion and Dataset Preparation: The proposed system begins with the ingestion of the BBC news dataset in CSV format, which contains raw news articles along with their

corresponding topic labels. Duplicate entries are removed to ensure data integrity and avoid biased learning. This dataset serves as the foundational input for the entire classification pipeline and enables supervised learning by providing labeled examples for training and evaluation.

Natural Language Processing (NLP) Preprocessing: In the second step, raw news text undergoes extensive NLP preprocessing to convert unstructured text into a clean and standardized format. This process includes converting text to lowercase, removing special characters and punctuation, tokenizing sentences into individual words, eliminating stop words, and applying lemmatization to reduce words to their base form. The preprocessing stage reduces noise, improves linguistic consistency, and enhances the effectiveness of subsequent feature extraction.

Label Encoding: After text preprocessing, the categorical topic labels are transformed into numerical form using label encoding. This conversion is essential for compatibility with machine learning algorithms, which require numerical target variables. The encoded labels represent fine-grained news categories such as business, entertainment, politics, sports, and technology.

Feature Extraction Using GloVe Word Embeddings: The cleaned text is then transformed into numerical feature vectors using pre-trained GloVe word embeddings with 100 dimensions. Each word in a news article is mapped to its corresponding GloVe vector, and document-level representations are generated by averaging all valid word vectors. This semantic embedding approach captures contextual relationships between words and enables

Exploratory Data Analysis (EDA): Exploratory Data Analysis is conducted to understand the statistical and linguistic characteristics of the dataset. Word clouds visualize frequently occurring terms, bar charts display common words and bigrams, histograms show document length distributions, and part-of-speech tagging highlights grammatical patterns. EDA helps validate preprocessing effectiveness and provides insights into class distribution and textual structure.

Data Splitting: The dataset is divided into training and testing subsets using an 80:20 ratio with stratified sampling. This ensures that all topic categories are proportionally represented

in both sets. The training data is used for learning model parameters, while the testing data is reserved for unbiased performance evaluation.

Training of Existing Baseline Models: In this step, multiple existing supervised classification models are trained using the GloVe-based feature vectors. These include the Radius Neighbors Classifier, Decision Tree Classifier, and Gaussian Naïve Bayes Classifier. Each model learns patterns in the semantic feature space and generates predictions for comparative performance analysis.

Proposed K-Nearest Neighbors (KNN) Model Training: The proposed methodology employs a K-Nearest Neighbors classifier as the primary model for fine-grained topic categorization. The KNN model uses Euclidean distance to identify the closest semantic neighbors for each news article and assigns the most frequent topic label among them. This distance-based approach effectively leverages semantic similarity captured by GloVe embeddings.

Model Evaluation and Performance Analysis: All trained models are evaluated using standard metrics such as accuracy, precision, recall, and F1-score. Confusion matrices are generated to analyze class-wise predictions, and multi-class AUC–ROC curves are plotted to assess discrimination capability. Performance comparison charts are used to identify the most effective classification model.

Prediction and Output Generation: Once evaluation is completed, the best-performing model is used to classify unseen news articles. The system supports both single-text prediction and batch-wise prediction from CSV files. Predicted numerical labels are mapped back to human-readable topic categories and presented as the final output.

Adaptive Learning and Model Reusability: The trained models are saved using joblib, allowing them to be reused without retraining. The system also supports retraining with updated datasets, enabling adaptation to evolving news content and vocabulary. This ensures long-term efficiency, scalability, and robustness of the proposed classification framework.

4.2 Preprocessing

The NLP preprocessing module serves as the foundation of the proposed news classification system by transforming raw, unstructured BBC news articles into a clean and meaningful

textual format. Since raw news content often contains noise such as punctuation, redundant words, and inconsistent formatting, preprocessing ensures linguistic uniformity and reduces irrelevant information. This step enhances the quality of feature extraction and enables machine learning models to learn semantic patterns more effectively.

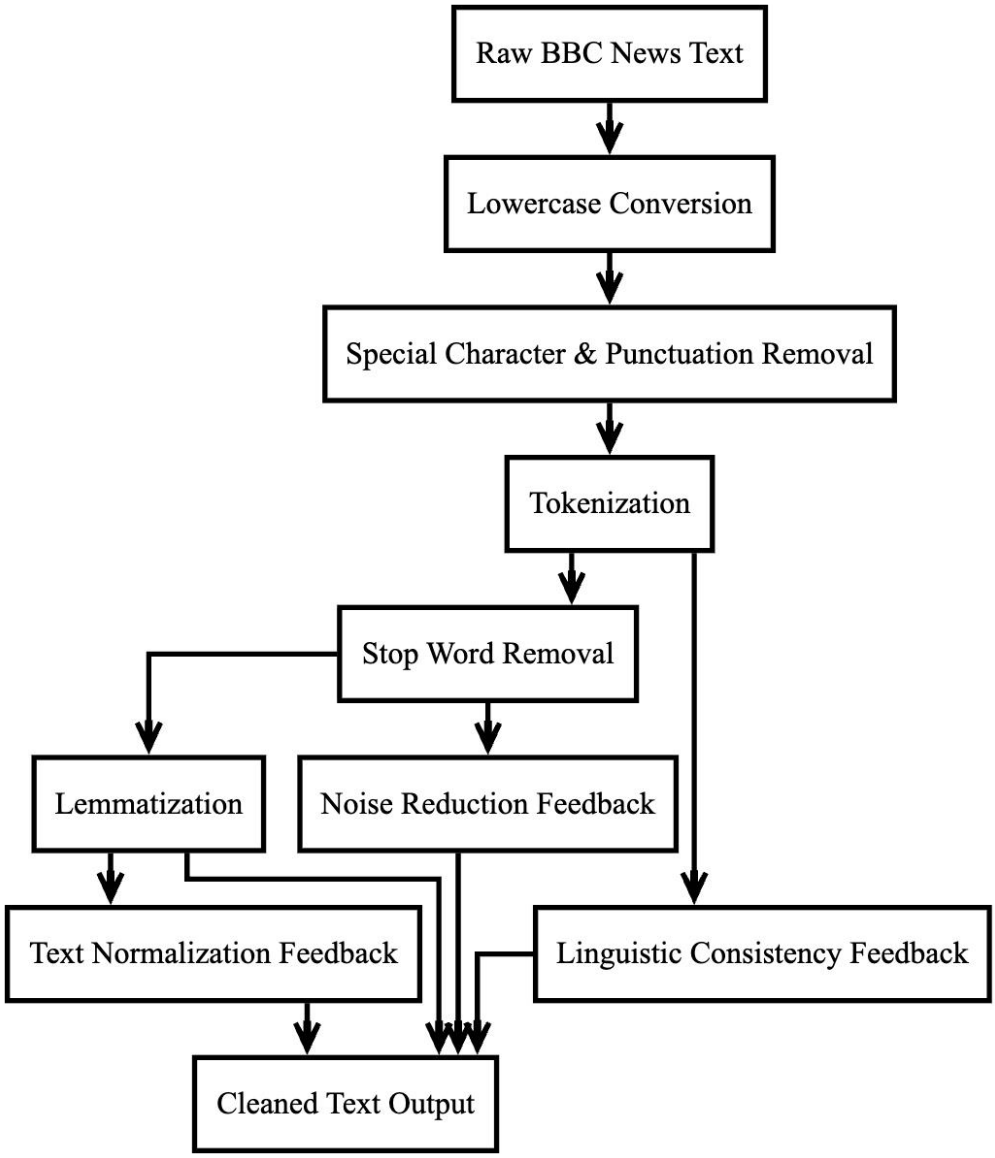


Fig. 4.2: Internal working of NLP Preprocessing.

Text Normalization: Each news article is first converted to lowercase to maintain uniformity across all textual data. This prevents the same word with different cases from being treated as separate tokens. Normalization simplifies text representation and improves consistency.

Removal of Special Characters and Noise: Special characters, punctuation marks, and unnecessary symbols are removed using regular expressions. This step eliminates non-informative tokens that do not contribute to semantic meaning. As a result, the text becomes cleaner and easier to analyze.

Tokenization: The cleaned text is split into individual words using tokenization. Tokenization breaks long sentences into manageable units that can be processed computationally. This allows further linguistic operations to be applied at the word level.

Stop Word Removal: Commonly occurring words such as “the”, “is”, and “and” are removed using a predefined stop-word list. These words usually do not add meaningful information to topic identification. Removing them reduces dimensionality and improves classification performance.

Lemmatization: Lemmatization converts words into their base or dictionary form. This ensures that different grammatical forms of a word are treated as the same feature. It helps preserve semantic meaning while reducing vocabulary size.

Clean Text Generation: After completing all preprocessing steps, the refined tokens are joined back into a clean text string. This final processed text is stored as a new feature and used for semantic feature extraction. It represents a noise-free and standardized version of the original article.

4.3 Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) plays a crucial role in understanding the linguistic structure, distribution, and characteristics of the BBC news dataset before model training. In this project, EDA is performed on the preprocessed textual data to analyze word usage patterns, document lengths, grammatical structure, and contextual relationships within the news articles. These analyses help validate the effectiveness of NLP preprocessing, identify dominant terms and themes, and ensure that the dataset is suitable for supervised document classification. By visualizing textual patterns, EDA provides valuable insights that guide feature extraction and model selection.

Token Collection and Word Frequency Analysis: In the first step, all cleaned news articles are tokenized into individual words, and a combined vocabulary is created. The frequency of each word is calculated using a counter mechanism. This helps identify the most commonly occurring terms across the entire dataset, revealing dominant topics and frequently discussed concepts within BBC news articles.

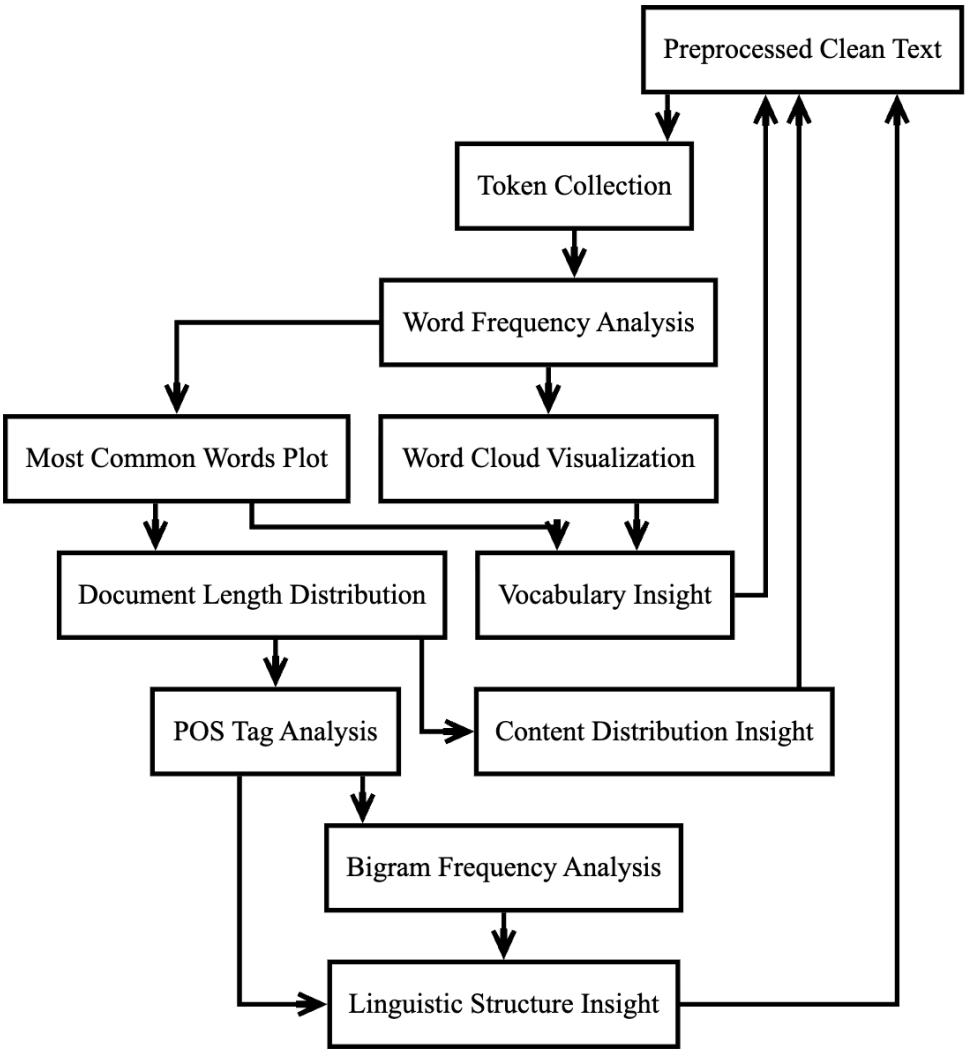


Fig. 4.3: Internal Working of EDA.

Word Cloud Visualization: A word cloud is generated using the most frequent words extracted from the dataset. This visual representation highlights prominent terms by displaying higher-frequency words in larger font sizes. The word cloud provides an intuitive overview of the dataset's thematic focus and confirms whether preprocessing has effectively removed irrelevant noise.

Most Common Words Distribution: The most frequently occurring words are visualized using a bar chart. This step quantitatively represents word dominance and helps compare

relative term frequencies. It assists in identifying meaningful keywords that contribute significantly to topic categorization.

Document Length Analysis: The length of each news article is calculated based on the number of tokens it contains. A histogram is plotted to show the distribution of document lengths across the dataset. This analysis helps determine variations in article size and ensures that extremely short or excessively long documents do not negatively affect model performance.

Part-of-Speech (POS) Tag Analysis: Part-of-speech tagging is applied to all tokens to analyze grammatical structures within the dataset. The frequency of POS tags such as nouns, verbs, adjectives, and adverbs is visualized using a bar plot. This step provides insight into the syntactic composition of news articles and supports effective feature representation.

Bigram Frequency Analysis: In the final step, bigrams (pairs of consecutive words) are generated from the tokenized text. The most frequent bigrams are visualized using a bar chart. This analysis captures contextual word relationships and common phrase patterns, which are valuable for understanding topic-specific language usage.

4.4 Glove Vector

GloVe (Global Vectors for Word Representation)–based feature extraction is used in this project to convert preprocessed BBC news articles into meaningful numerical representations that preserve semantic relationships between words. Unlike traditional bag-of-words models, GloVe embeddings capture contextual similarity by leveraging global word co-occurrence statistics learned from large corpora. By representing each news article as a dense, fixed-length vector, the system enables machine learning classifiers to effectively measure similarity and perform fine-grained topic categorization.

Loading Pre-trained GloVe Embeddings: In the first step, pre-trained GloVe word embeddings with 100 dimensions are loaded from the external file (glove.6B.100d.txt). Each word is mapped to a dense numerical vector, creating a lookup dictionary that allows rapid retrieval of word embeddings during feature generation.

Handling Missing or Invalid Text: Before vectorization, the cleaned text column is checked for missing or null values. Any empty or non-string entries are replaced with empty strings to ensure stable and error-free processing during vector conversion.

Token Extraction from Cleaned Text: Each cleaned news article is split into individual words. These tokens represent the fundamental units for embedding lookup. Only valid tokens that exist in the GloVe vocabulary are considered, ensuring meaningful semantic representation.

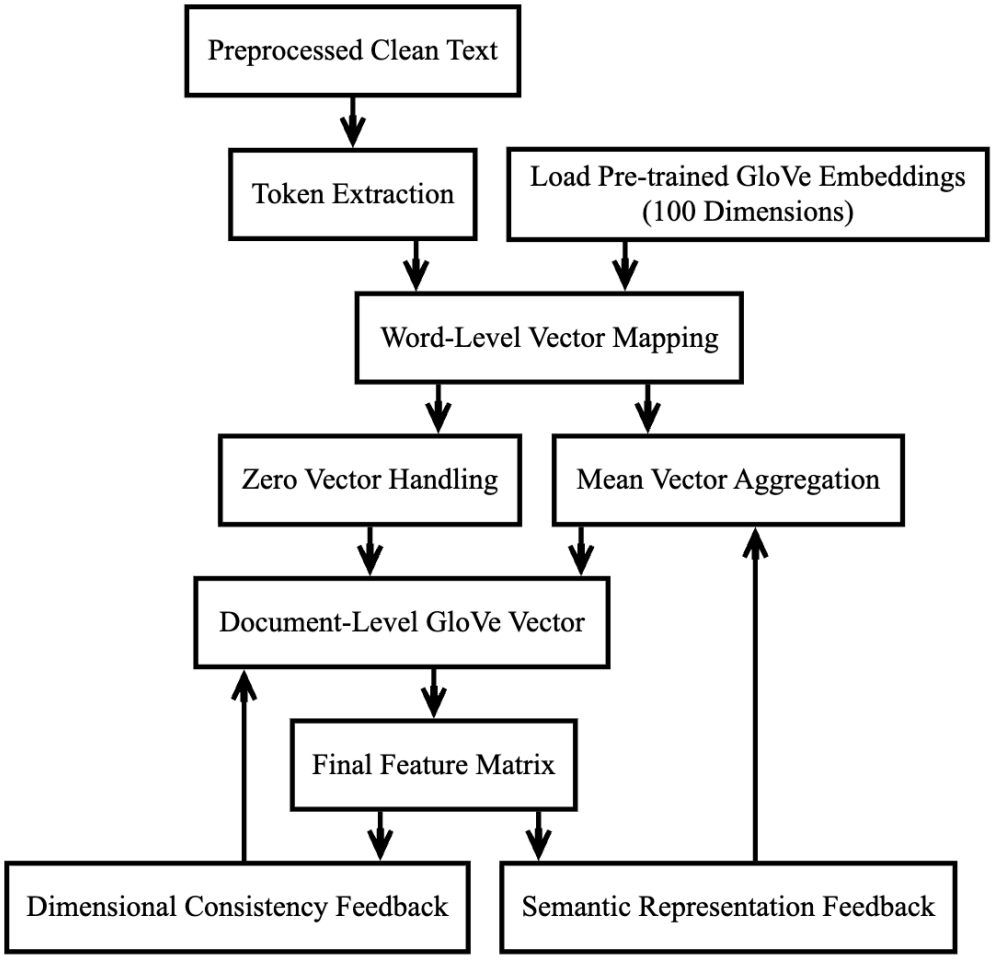


Fig. 4.4: Internal Working flow of Glove Vector.

Word-Level Vector Mapping: For every token in a news article, the corresponding 100-dimensional GloVe vector is retrieved from the embedding dictionary. Words that are not present in the GloVe vocabulary are ignored, preventing noise and unreliable representations from affecting the document vector.

Document-Level Vector Construction: All valid word vectors belonging to a single news article are aggregated by computing their mean. This averaging process produces a single fixed-length vector that represents the entire document, capturing its overall semantic meaning.

Zero-Vector Handling for Empty Documents: If a document contains no valid GloVe-mapped words, a zero vector of length 100 is assigned. This ensures consistent feature dimensions across all samples and prevents computational errors during model training.

Feature Matrix Generation: The document-level GloVe vectors are stacked into a numerical feature matrix using NumPy. This matrix serves as the final input for supervised learning models, enabling efficient training, evaluation, and prediction.

4.5 Train Test Split

The train–test split is a critical stage in the proposed classification framework, as it ensures reliable evaluation of supervised learning models on unseen data. After converting BBC news articles into fixed-length GloVe feature vectors and encoding their corresponding labels, the dataset is partitioned into training and testing subsets. This separation allows the models to learn patterns from the training data while being evaluated on independent test data, thereby preventing overfitting and ensuring fair performance assessment.

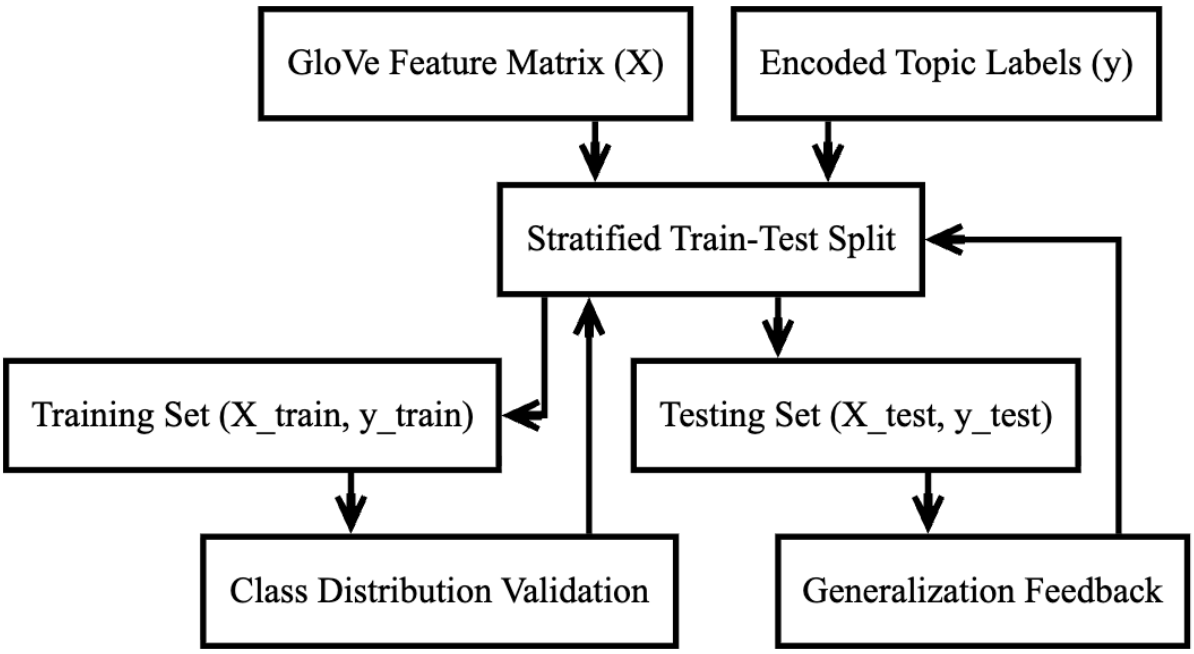


Fig. 4.5: Internal Working flow of Train test splitting.

Feature and Target Separation: In the first step, the extracted GloVe vectors are stacked into a numerical feature matrix, while the encoded topic labels are stored as a separate target vector. This clear separation of input features and output labels ensures compatibility with machine learning model training and evaluation processes.

Stratified Sampling Strategy: To preserve the original class distribution across all topic categories, stratified sampling is applied during data splitting. This ensures that each category, such as business, politics, sports, entertainment, and technology, is proportionally represented in both training and testing datasets.

Training and Testing Ratio Selection: The dataset is divided using an 80:20 ratio, where 80 percent of the data is allocated for training and 20 percent is reserved for testing. This ratio provides sufficient data for model learning while retaining an adequate sample size for unbiased evaluation.

Random State Initialization: A fixed random state value is used during the split process to ensure reproducibility of results. This guarantees that the same training and testing partitions are generated across multiple executions, enabling consistent model comparison and experimentation.

Generation of Training and Testing Sets: As a result of the splitting process, four distinct datasets are produced: training features, testing features, training labels, and testing labels. These datasets are subsequently used for model training, validation, and performance evaluation within the proposed framework.

4.6 Model Building &Training

The model building and training phase forms the core of the proposed fine-grained BBC news topic categorization system, where semantic feature representations obtained from GloVe embeddings are utilized to train multiple supervised learning classifiers. After splitting the dataset into stratified training and testing sets, the training data is fed into a set of existing baseline models, namely Radius Neighbors Classifier, Decision Tree Classifier, and Gaussian Naïve Bayes, to establish comparative performance benchmarks. In parallel, a K-Nearest Neighbors (KNN) classifier is implemented as the proposed model, leveraging distance-based similarity in the semantic embedding space to assign topic labels. Each model learns underlying patterns from the training data and is optimized using default hyperparameters suitable for text-based feature vectors. The trained models are serialized using joblib for reuse and reproducibility. This multi-model training strategy enables robust comparison, ensures reliable learning from high-dimensional semantic features, and lays the foundation for accurate topic prediction and performance evaluation.

4.6.1 Radius Neighbors Classifier

The Radius Neighbors Classifier operates as a distance-based supervised learning model that classifies BBC news articles by examining the local neighborhood of semantically similar documents within a predefined radius. In the proposed fine-grained topic categorization system, each news article is represented as a dense GloVe-based feature vector, allowing the classifier to measure semantic similarity using Euclidean distance. Unlike fixed-k neighbor approaches, this model dynamically considers all neighboring documents that fall within the specified radius, making it suitable for capturing localized semantic patterns across diverse news topics. By leveraging proximity in the embedding space, the Radius Neighbors Classifier effectively assigns topic labels based on contextual similarity between articles.

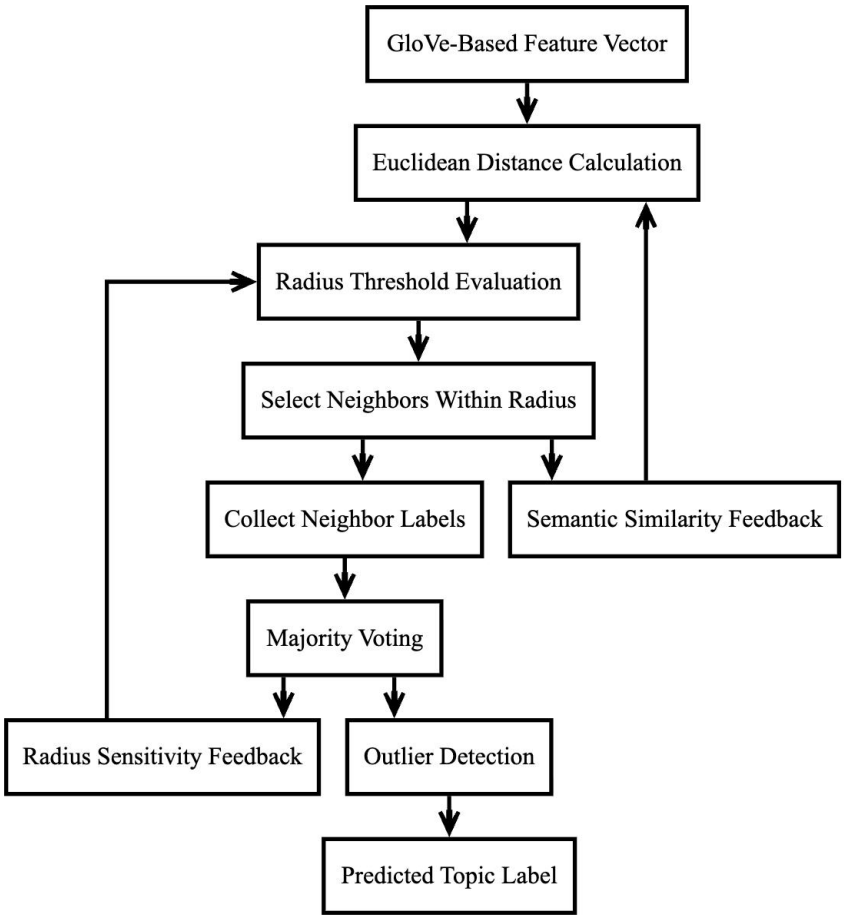


Fig. 4.6: Internal working of Radius Neighbors classifier.

Input Feature Reception: The classifier receives the GloVe-embedded feature vectors generated from preprocessed BBC news articles. Each vector represents the semantic

meaning of an entire document in a fixed-dimensional space. These vectors form the basis for all distance calculations during training and prediction.

Distance Metric Application: For each test instance, the classifier computes the Euclidean distance between the input vector and all training vectors. This distance measurement quantifies semantic similarity, where smaller distances indicate higher contextual relevance between news articles.

Radius-Based Neighbor Selection: Instead of selecting a fixed number of neighbors, the classifier identifies all training samples that lie within the predefined radius threshold. This adaptive neighborhood selection allows the model to flexibly adjust to variations in data density across different topic clusters.

Label Aggregation within Radius: Once the neighbors within the radius are identified, the classifier collects their corresponding topic labels. A majority voting mechanism is applied, where the most frequent label among the selected neighbors is chosen as the predicted category for the input article.

Outlier Handling Mechanism: If no neighbors are found within the specified radius, the classifier assigns a default outlier label. This mechanism ensures robustness by preventing undefined predictions for isolated or unique news articles.

Final Topic Assignment: The selected topic label is mapped back to its human-readable form, such as business, politics, sports, entertainment, or technology. This final output represents the predicted fine-grained category for the given BBC news article.

Drawbacks

The Radius Neighbors Classifier did not achieve high accuracy or consistently accurate predictions in this project primarily due to the high-dimensional and sparsely distributed nature of GloVe-based text embeddings. In fine-grained BBC news topic categorization, semantic clusters often overlap, making it difficult to define an optimal radius that works uniformly across all categories. As a result, the classifier either includes irrelevant neighbors or fails to find sufficient neighbors, leading to unstable and inaccurate predictions.

Sensitivity to Radius Selection: The performance of the Radius Neighbors Classifier heavily depends on the choice of radius value. A small radius may result in no neighbors being selected, while a large radius may include semantically unrelated articles, reducing classification accuracy.

Poor Performance in High-Dimensional Feature Space: GloVe embeddings generate 100-dimensional feature vectors, where distance-based similarity becomes less meaningful due to the curse of dimensionality. This reduces the effectiveness of Euclidean distance in accurately capturing semantic similarity.

Inconsistent Predictions Across Classes: Different news categories have varying data densities. The radius-based approach struggles to adapt to these variations, leading to biased predictions for dense classes and weak performance for sparse ones.

Outlier Sensitivity: Articles that lie outside dense clusters are often treated as outliers, resulting in incorrect or default label assignments. This negatively impacts overall classification reliability.

Computational Inefficiency: For large datasets, the classifier must compute distances between the test instance and all training samples. This increases prediction time and makes the model less suitable for real-time applications.

Lack of Adaptability to Text Semantics: Radius-based classification relies purely on geometric distance and does not inherently adapt to contextual nuances in language, limiting its effectiveness in fine-grained text classification tasks.

4.6.2 Decision Tree classifier

The Decision Tree Classifier operates as a rule-based supervised learning model that categorizes BBC news articles by learning hierarchical decision rules from training data. In the proposed fine-grained topic categorization system, each news article is represented using GloVe-based semantic feature vectors, and the decision tree recursively splits this feature space to separate different topic categories. The model constructs an interpretable tree structure where internal nodes represent decision conditions, branches represent outcomes of those conditions, and leaf nodes correspond to final topic labels. This approach enables the classifier to model non-linear relationships between textual features and news categories.

Input Feature Representation: The classifier receives the GloVe-generated feature vectors obtained from preprocessed BBC news articles. Each vector represents the semantic composition of a document in numerical form and serves as the input for tree construction and prediction.

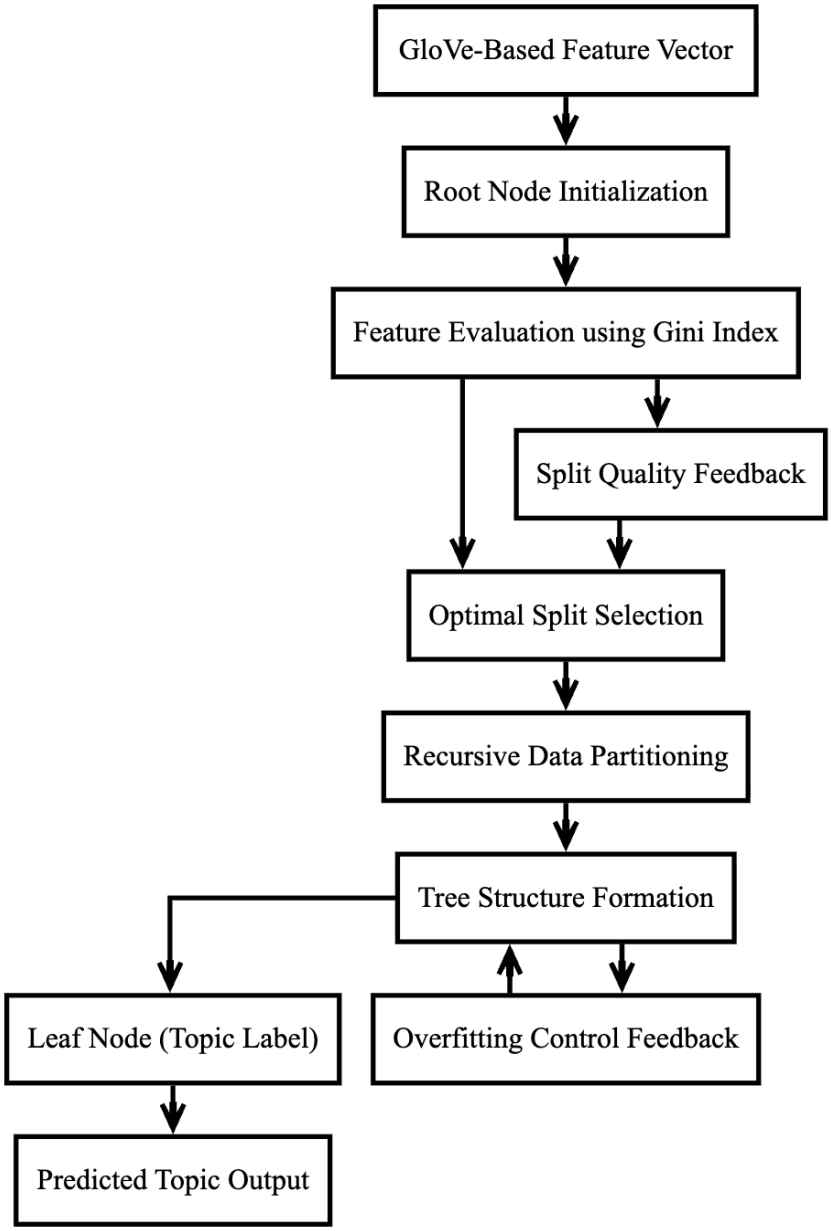


Fig. 4.7: Internal Working of Decision Tree classifier.

Feature Selection for Node Splitting: At each node of the tree, the algorithm evaluates all available features to determine the best split. The Gini impurity criterion is used to measure how well a feature separates the data into distinct topic categories, selecting the feature that minimizes impurity.

Recursive Data Partitioning: Once the optimal feature and split threshold are identified, the dataset is divided into subsets. This recursive partitioning continues for each child node, progressively refining the separation between different news topics.

Tree Growth and Structure Formation: The tree continues to grow until stopping conditions are met, such as all samples in a node belonging to the same class or no further impurity reduction being possible. This results in a hierarchical structure that captures decision rules for topic classification.

Leaf Node Label Assignment: At the terminal nodes (leaf nodes), a topic label is assigned based on the majority class of training samples within that node. These labels represent the final decision outcomes of the classifier.

Prediction Process for New Articles: During prediction, a new BBC news article's feature vector traverses the tree from the root to a leaf node by following the learned decision rules. The topic label associated with the reached leaf node is returned as the predicted category.

Drawbacks

The Decision Tree Classifier failed to achieve consistently high accuracy in this project mainly due to the high-dimensional nature of GloVe-based text features and the complex overlap between fine-grained news topics. Decision trees tend to overfit training data when handling continuous, dense feature vectors, leading to reduced generalization on unseen BBC news articles. As a result, the model struggled to maintain stable and accurate predictions across all topic categories.

Overfitting on Training Data: Decision trees often create overly complex structures when trained on high-dimensional feature spaces. This causes the model to memorize training samples rather than learning generalizable patterns, resulting in poor performance on test data.

Sensitivity to Small Data Variations: Minor changes in training data can significantly alter the structure of the tree. This instability leads to inconsistent predictions, especially when dealing with noisy or semantically similar news articles.

Ineffective Handling of Continuous Embeddings: GloVe vectors are continuous-valued features, and decision trees rely on threshold-based splits that may not capture subtle semantic relationships effectively. This limits their ability to model contextual similarities in text data.

Bias Toward Dominant Classes: Decision trees tend to favor classes with higher sample counts, which can cause biased predictions toward frequently occurring news categories while underperforming on minority classes.

Limited Scalability: As dataset size increases, the tree grows deeper and more complex, increasing training time and memory usage. This makes the model less suitable for large-scale text classification tasks.

Lower Robustness to Noise: Textual data inherently contains semantic noise and overlapping concepts. Decision trees lack inherent mechanisms to smooth such noise, which negatively impacts classification accuracy.

4.6.3 Gaussian Naïve Bayes

The Gaussian Naïve Bayes classifier operates as a probabilistic supervised learning model that categorizes BBC news articles by applying Bayes' theorem with the assumption of feature independence. In the proposed fine-grained topic categorization system, each news article is represented using continuous-valued GloVe-based feature vectors. The model assumes that these features follow a Gaussian (normal) distribution within each topic class. By estimating class-wise statistical parameters, Gaussian Naïve Bayes computes the likelihood of a document belonging to each topic and assigns the category with the highest posterior probability.

Input Feature Representation: The classifier receives GloVe-based document vectors generated from preprocessed BBC news articles. Each vector contains continuous numerical values representing the semantic composition of the text and serves as the input for probabilistic modeling.

Class Prior Probability Estimation: During training, the model calculates prior probabilities for each news category based on their frequency in the training dataset. These priors represent the likelihood of each topic occurring before considering the input features.

Feature Distribution Modeling: For every topic class, the classifier computes the mean and variance of each feature dimension across all training samples. These statistical parameters define the Gaussian distribution assumed for each feature within a class.

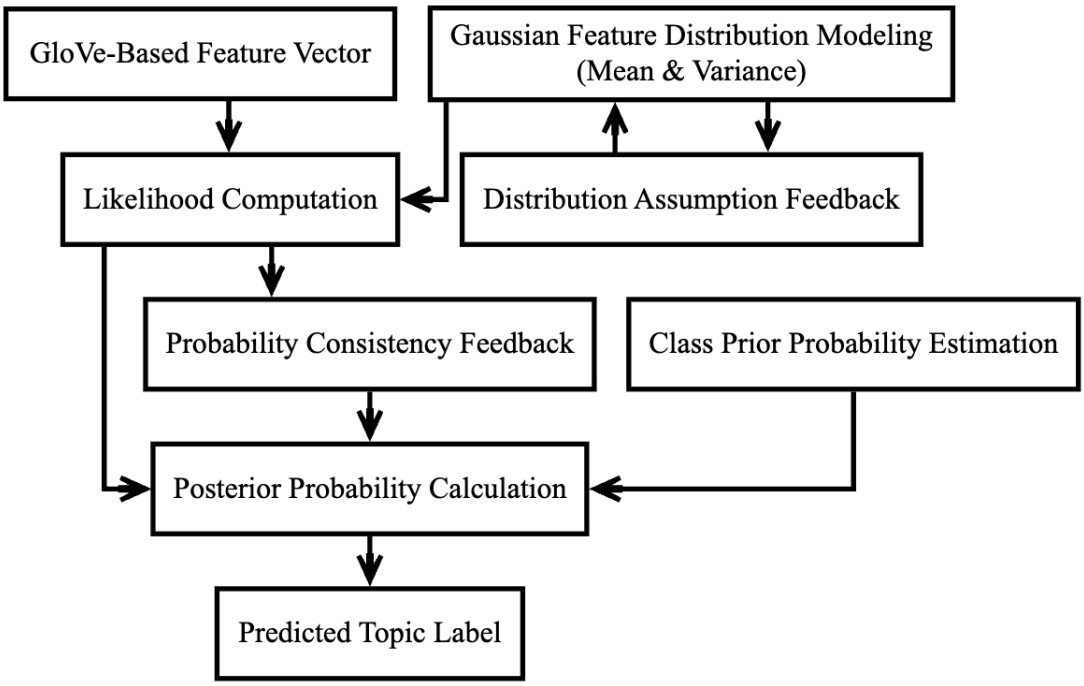


Fig. 4.8: Internal Working of Gaussian naïve bayes.

Likelihood Computation: When a new article is evaluated, the model computes the likelihood of each feature value under the Gaussian distribution of every class. These likelihoods quantify how well the document's features fit each topic category.

Posterior Probability Calculation: The computed likelihoods are combined with the class prior probabilities using Bayes' theorem to calculate posterior probabilities for each topic. This step determines the overall probability of the document belonging to each category.

Final Topic Selection: The topic label corresponding to the highest posterior probability is selected as the predicted category for the BBC news article. This label represents the model's final classification output.

Drawbacks

The Gaussian Naïve Bayes classifier failed to achieve high accuracy in this project primarily due to its strong assumptions of feature independence and Gaussian distribution, which do

not hold for GloVe-based text embeddings. In fine-grained BBC news topic categorization, semantic features are highly correlated and context-dependent, causing the probabilistic estimates to be inaccurate. As a result, the model struggled to capture complex relationships between words and topics, leading to reduced prediction reliability.

Unrealistic Feature Independence Assumption: Gaussian Naïve Bayes assumes that all features are independent given the class label. In textual data represented by word embeddings, features are inherently correlated, which violates this assumption and degrades classification performance.

Poor Fit to High-Dimensional Embeddings: GloVe vectors are dense and high-dimensional, making it difficult for Gaussian distributions to accurately model feature behavior. This mismatch results in unreliable likelihood estimations for topic classification.

Sensitivity to Data Distribution: The model assumes a normal distribution for each feature within a class. However, semantic feature values often exhibit non-Gaussian patterns, leading to incorrect probability calculations.

Limited Contextual Understanding: Gaussian Naïve Bayes treats each feature independently and does not capture contextual or sequential relationships between words, which are crucial for fine-grained topic categorization.

Bias Toward Dominant Classes: When class distributions are imbalanced, the prior probability component can dominate predictions, causing bias toward frequently occurring news categories.

Lower Robustness to Noise: Text data contains semantic ambiguity and noise, which negatively impacts probabilistic estimation. Gaussian Naïve Bayes lacks built-in mechanisms to handle such noise effectively.

4.6.4 KNN Classifier

The K-Nearest Neighbors (KNN) classifier operates as a distance-based supervised learning model that categorizes BBC news articles by identifying the most semantically similar documents in the training dataset. In the proposed fine-grained topic categorization system, each news article is represented using GloVe-based feature vectors, enabling the classifier to

compute similarity using Euclidean distance. Unlike probabilistic or rule-based models, KNN does not build an explicit model during training; instead, it relies on instance-based learning, where classification decisions are made at prediction time based on the proximity of feature vectors. This makes KNN particularly effective for capturing semantic similarity between news articles belonging to the same topic.

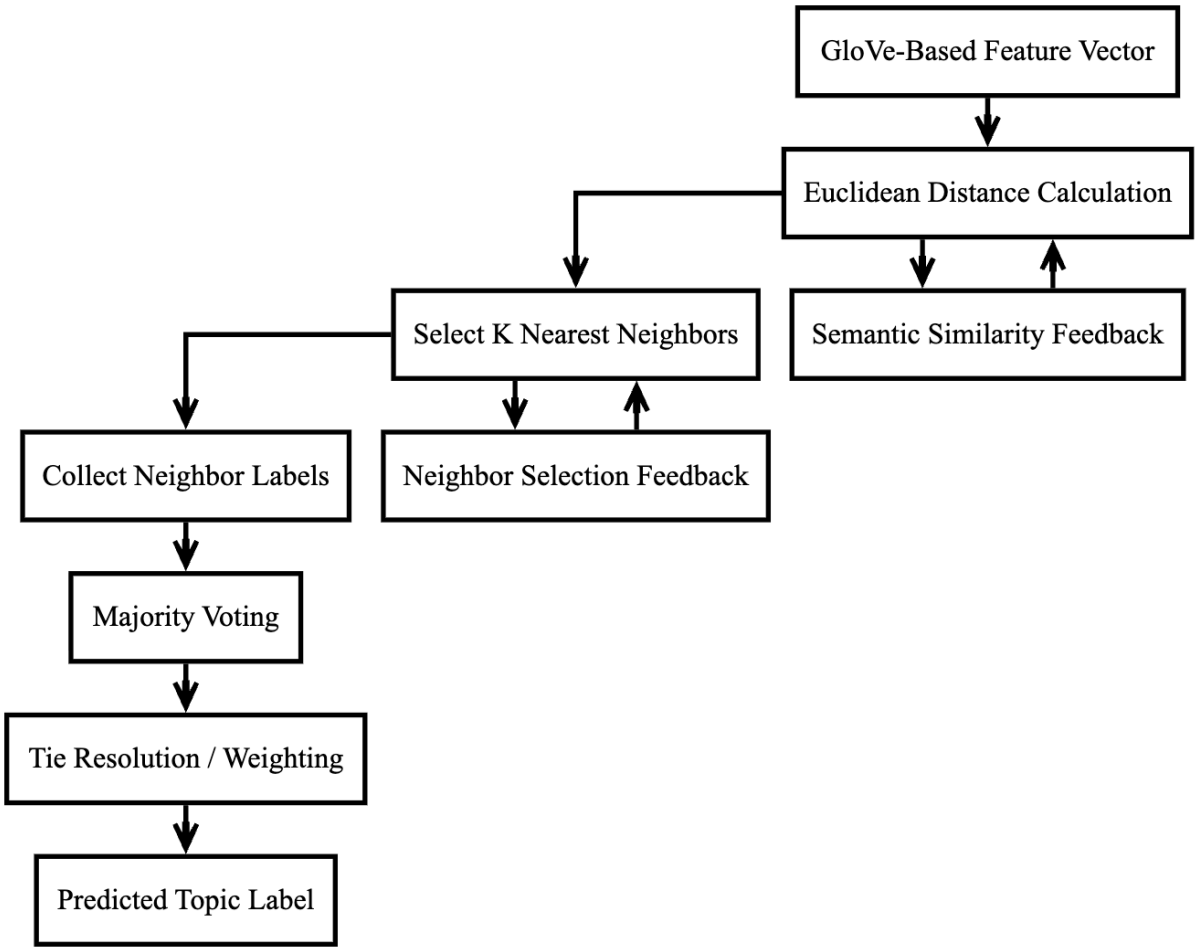


Fig. 4.9: Internal Working of KNN Classifier.

Feature Vector Input: The classifier receives the GloVe-embedded document vectors obtained from preprocessed BBC news articles. Each vector represents the semantic meaning of a news article in a fixed-dimensional numerical space and serves as the input for similarity comparison.

Distance Computation: For a given test article, the KNN classifier calculates the Euclidean distance between the input vector and all vectors in the training dataset. This distance metric quantifies how semantically close the test article is to previously seen news articles.

Neighbor Selection: Based on the computed distances, the classifier selects the K nearest neighbors, where K is a predefined parameter. These neighbors represent the most similar news articles in terms of semantic content.

Label Aggregation: The topic labels associated with the selected neighbors are collected. A majority voting mechanism is applied, where the most frequently occurring label among the neighbors is chosen as the predicted category.

Tie Resolution and Weighting: In cases where multiple labels have equal votes, distance-based weighting or predefined tie-breaking strategies are applied. This ensures stable and deterministic classification outcomes.

Final Topic Prediction: The selected topic label is mapped back to its human-readable form, such as business, politics, sports, entertainment, or technology. This label represents the final prediction for the given BBC news article.

Advantages

The K-Nearest Neighbors classifier achieved good accuracy and reliable predictions in this project because it directly leverages semantic similarity captured by GloVe word embeddings. By comparing each test news article with the most similar training articles, KNN effectively grouped semantically related content into the same topic category. Its instance-based learning approach allowed it to adapt naturally to overlapping and fine-grained BBC news topics without relying on strict assumptions.

Effective Use of Semantic Similarity: KNN performs classification based on distance in the embedding space, making it highly suitable for GloVe-based representations. Semantically similar news articles tend to lie closer together, enabling accurate topic assignment.

No Assumption About Data Distribution: Unlike Gaussian Naïve Bayes, KNN does not assume any specific data distribution. This flexibility allows it to handle complex, non-linear relationships present in text embeddings.

Strong Performance for Fine-Grained Classification: KNN excels in scenarios where topic boundaries overlap. By relying on local neighborhood information, it accurately captures subtle differences between closely related news topics.

Adaptability to Diverse Data Patterns: The classifier naturally adapts to varying data densities across classes. It can perform well even when some news categories have more samples than others.

Simplicity and Interpretability: KNN is easy to understand and implement. Predictions can be intuitively explained by examining the nearest neighboring articles that influenced the decision.

Robustness to Noise After Preprocessing: When combined with effective NLP preprocessing and GloVe embeddings, KNN becomes less sensitive to irrelevant features, leading to stable and consistent predictions.

No Explicit Training Phase: Since KNN does not require model training, it avoids overfitting issues associated with complex models. This makes it a reliable choice for datasets where semantic similarity plays a dominant role.

CHAPTER 5

UML DIAGRAMS

Unified Modeling Language (UML) is a standardized visual language used to model, design, and document the architecture of software systems. It provides a set of graphical notations to represent the structure and behavior of a system, making complex systems easier to understand and communicate among developers, stakeholders, and business analysts.

Key Points About UML

- **Standardized Notation:** UML offers a universal set of symbols and diagrams that standardize how software systems are described, which helps teams speak the same language regardless of their background or the programming language they use.
- **Types of Diagrams:** UML includes various diagrams that can be categorized into two main types:
    - **Structural Diagrams:** These describe the static aspects of the system. Examples include Class Diagrams, Component Diagrams, and Deployment Diagrams.
    - **Behavioral Diagrams:** These focus on the dynamic aspects and interactions within the system. Examples include Use Case Diagrams, Sequence Diagrams, Activity Diagrams, and Collaboration Diagrams.
- **System Documentation and Communication:** UML serves as an effective tool for documenting system requirements, design decisions, and the overall architecture. It helps bridge the gap between technical and non-technical stakeholders by providing clear, visual representations of the system.
- **Design and Analysis:** By modeling different aspects of a system, UML enables developers to analyze and validate the design early in the development process. This can lead to better decision-making, reduced complexity, and improved system maintainability.
- **Flexibility:** UML is versatile and can be used across a wide range of applications, from small-scale projects to large, complex systems. It supports object-oriented design principles and can be adapted to various methodologies such as Agile or Waterfall.

5.1 Class Diagram

The class diagram represents the structural organization of the fine-grained BBC news topic categorization system by illustrating how different software components interact to achieve supervised document classification. It captures the flow from data ingestion and NLP preprocessing to feature extraction using GloVe vectors and subsequent model training and prediction. Core classes such as DatasetLoader, TextPreprocessor, GloveVectorizer, and ModelTrainer work together to transform raw BBC news articles into meaningful numerical representations and manage the end-to-end classification pipeline.

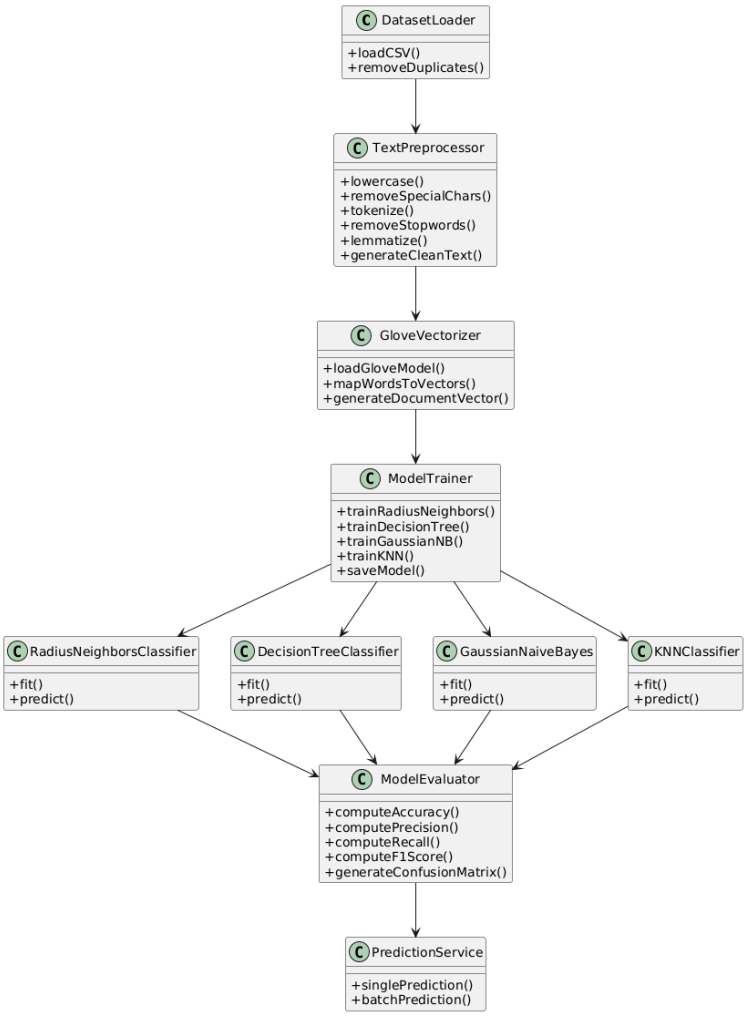


Fig. 5.1: Class Diagram.

The diagram also highlights the role of multiple classification algorithms within the system. Separate classifier classes are defined for RadiusNeighborsClassifier, DecisionTreeClassifier, GaussianNaiveBayes, and KNNClassifier, all coordinated through a common ModelTrainer

and ModelEvaluator interface. This design ensures modularity, making it easy to compare model performance, evaluate results using metrics, and perform single or batch-wise predictions. By explicitly representing all algorithms in the UML structure, the class diagram emphasizes clarity, extensibility, and maintainability of the overall system architecture.

5.2 Use Case Diagram

The use case diagram illustrates how a user interacts with the fine-grained BBC news topic categorization system to perform end-to-end document classification tasks. It captures user-driven operations such as uploading the BBC news dataset, performing NLP preprocessing, extracting semantic features using GloVe vectors, and initiating model training. Through these interactions, the user can evaluate and compare the performance of multiple supervised classifiers, including Radius Neighbors Classifier, Decision Tree Classifier, Gaussian Naïve Bayes, and the proposed K-Nearest Neighbors (KNN) classifier.

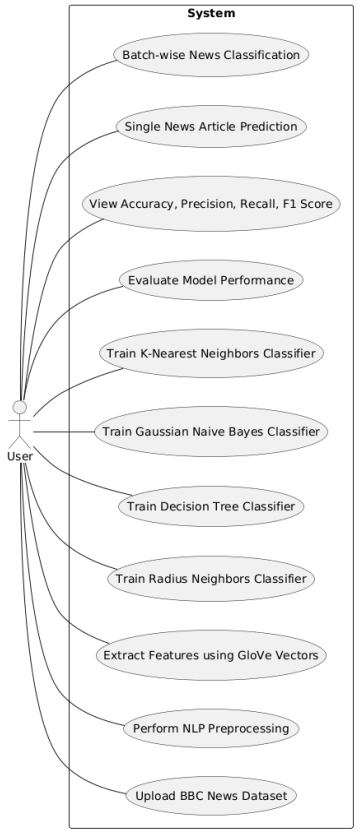


Fig. 5.2: Use Case Diagram.

The diagram also represents prediction-oriented use cases where the user can perform single-text prediction or batch-wise classification on unseen BBC news articles. After model

execution, the user can view evaluation metrics such as accuracy, precision, recall, and F1-score generated by each algorithm. This UML representation emphasizes user satisfaction by clearly defining system functionality, ensuring transparency in algorithm usage, and providing an intuitive interaction flow between the user and all implemented classification models.

5.3 Data Flow Diagram

The data flow diagram represents how data moves through the fine-grained BBC news topic categorization system from input to final prediction. It shows how raw BBC news articles provided by the user are processed through NLP preprocessing and GloVe-based feature extraction before being forwarded to the supervised learning layer. The diagram clearly depicts how the same processed feature set is shared across multiple classification algorithms, ensuring uniform and fair model comparison while maintaining a smooth and transparent data pipeline.

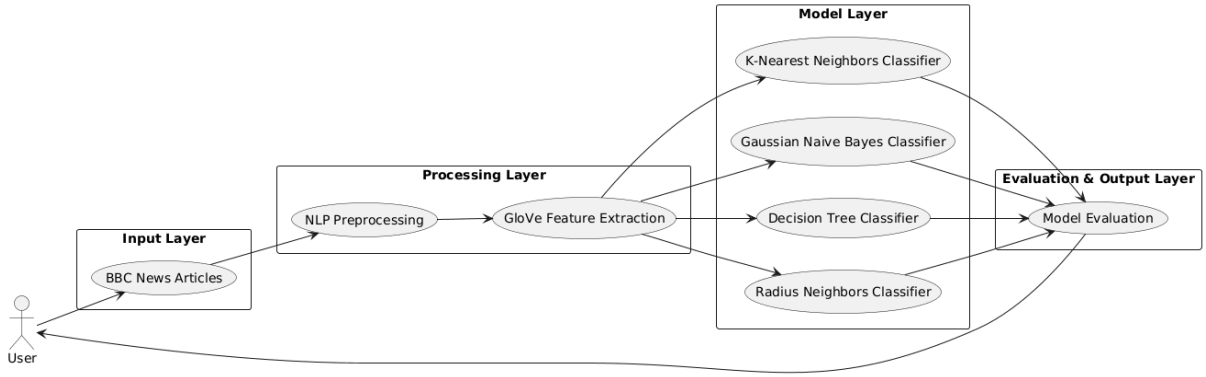


Fig. 5.3: Data Flow Diagram.

The diagram also illustrates how predictions and evaluation results flow back to the user after classification. Radius Neighbors Classifier, Decision Tree Classifier, Gaussian Naïve Bayes, and the proposed K-Nearest Neighbors (KNN) classifier independently process the extracted features to generate topic predictions. These outputs are then evaluated using performance metrics and returned to the user for analysis, enabling both single and batch-wise prediction while ensuring clarity in how data is transformed and utilized at each stage.

5.4 Deployment Diagram

The deployment diagram illustrates how the fine-grained BBC news topic categorization system is physically deployed across hardware and software environments to support execution, training, and prediction. It shows the interaction between the user's local machine, the Python execution environment, and the underlying libraries that handle NLP preprocessing, GloVe-based feature extraction, and supervised learning. The architecture ensures that raw BBC news articles are processed efficiently and that all classification algorithms operate within a unified runtime setup.

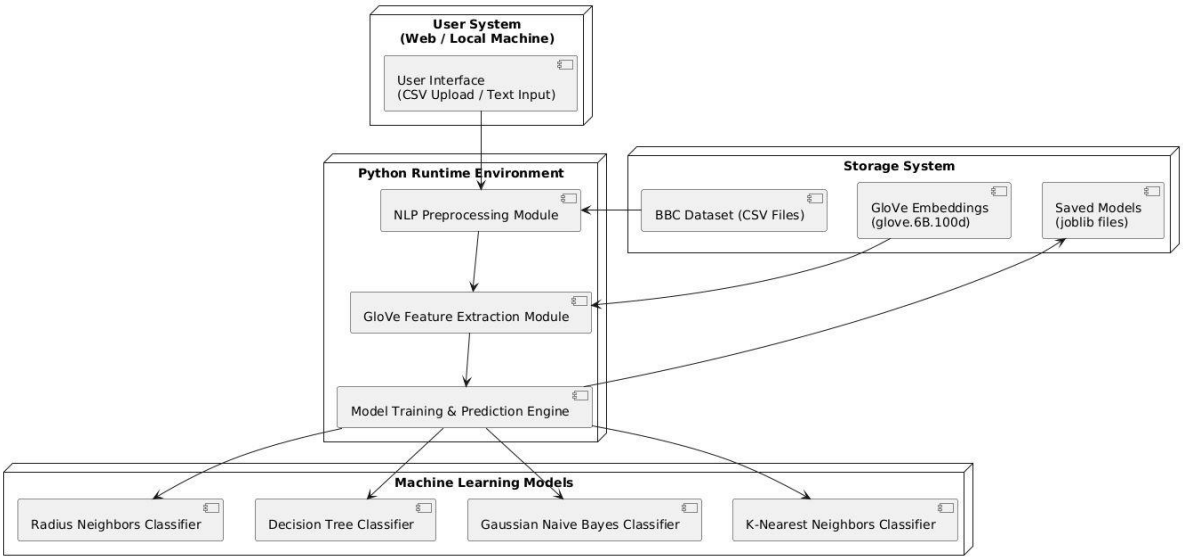


Fig. 5.4: Deployment Diagram.

The diagram also emphasizes how trained models such as Radius Neighbors Classifier, Decision Tree Classifier, Gaussian Naïve Bayes, and K-Nearest Neighbors (KNN) are deployed and executed within the machine learning layer. Pre-trained GloVe embeddings and saved model files are accessed from storage during runtime to support both training and prediction. This deployment structure enables seamless batch-wise and single-article classification while ensuring scalability, portability, and ease of maintenance for the overall system.

5.5 Component Diagram

The component diagram illustrates the modular organization of the fine-grained BBC news topic categorization system by showing how independent functional components interact to deliver end-to-end document classification. It highlights how raw news data flows through

dedicated components for NLP preprocessing and GloVe-based feature extraction before reaching the supervised learning layer. Each component is designed to handle a specific responsibility, ensuring loose coupling, reusability, and ease of system maintenance.

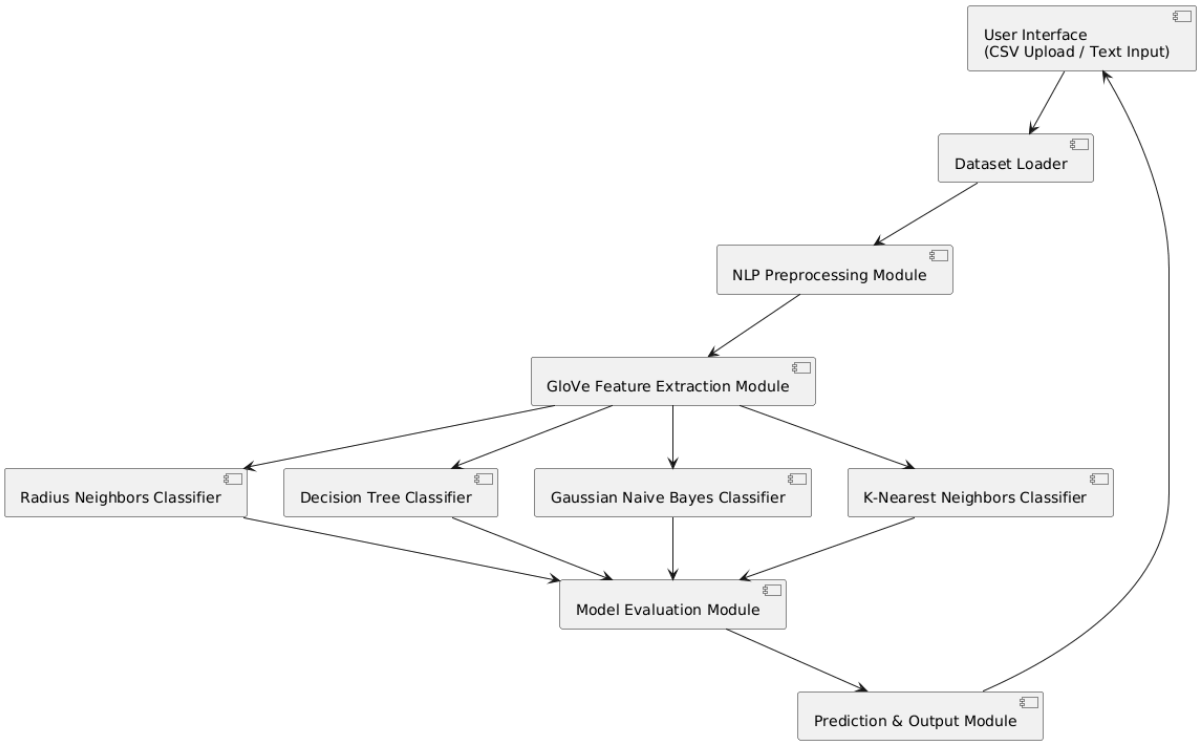


Fig. 5.5: Component Diagram.

The diagram also emphasizes the role of multiple classification components working in parallel. Separate components are defined for Radius Neighbors Classifier, Decision Tree Classifier, Gaussian Naïve Bayes, and K-Nearest Neighbors (KNN), all coordinated through a central model management and evaluation component. This structure enables seamless comparison of algorithm performance, supports single and batch-wise prediction, and ensures that results are consistently delivered to the user, thereby improving transparency and overall user satisfaction.

5.6 Collaboration Diagram

The collaboration diagram explains how different objects in the fine-grained BBC news topic categorization system interact with one another through message passing to accomplish supervised document classification. It focuses on runtime communication between components such as data loading, NLP preprocessing, GloVe feature extraction, and model

execution. The diagram highlights how control flows sequentially from the user request to the processing modules, ensuring coordinated execution of each operation.

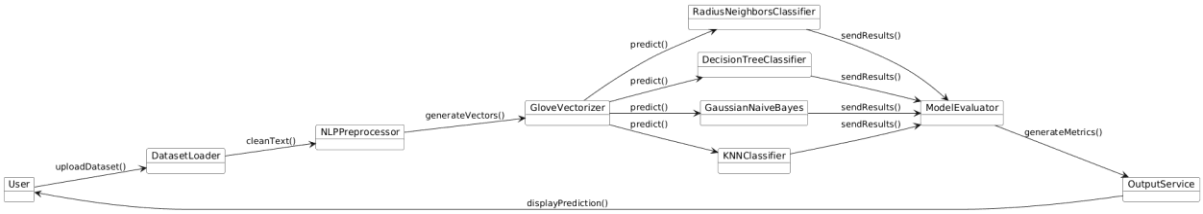


Fig. 5.6: Collaboration Diagram.

The interaction further demonstrates how the extracted GloVe feature vectors are shared across all classification algorithms, including Radius Neighbors Classifier, Decision Tree Classifier, Gaussian Naïve Bayes, and K-Nearest Neighbors (KNN). Each classifier communicates its prediction results to the evaluation module, which aggregates performance metrics before sending the final prediction back to the user. This collaborative interaction ensures consistency, efficient coordination among algorithms, and transparent decision-making within the system.

5.7 Sequence Diagram

The sequence diagram describes the time-ordered interaction between system components involved in fine-grained BBC news topic categorization. It shows how a user request initiates a sequential flow starting from dataset submission or text input, followed by NLP preprocessing and GloVe-based feature extraction. This ordered execution ensures that raw news articles are systematically transformed into semantic feature vectors before being passed to the classification stage.

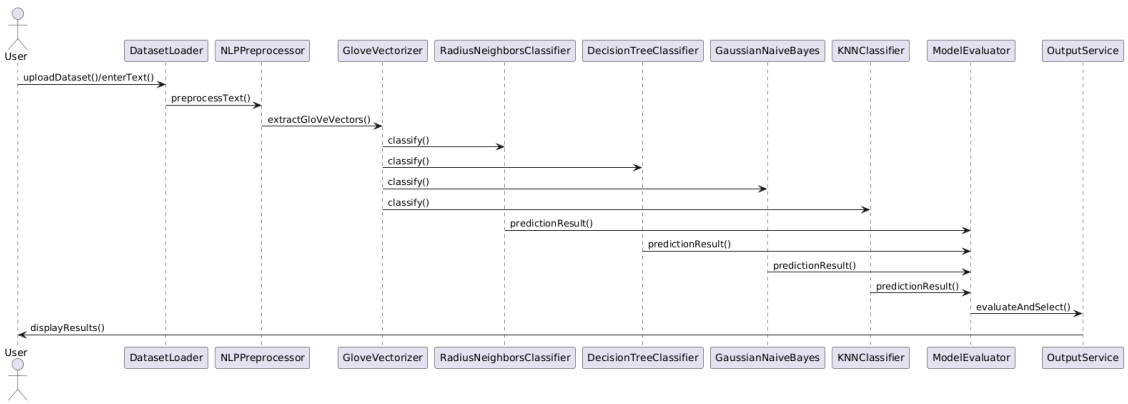


Fig. 5.7: Sequence Diagram.

The diagram further illustrates how the extracted features are sequentially supplied to multiple supervised classifiers, namely Radius Neighbors Classifier, Decision Tree Classifier, Gaussian Naïve Bayes, and K-Nearest Neighbors (KNN). Each classifier processes the input and sends its prediction to the evaluation module, which computes performance metrics and selects the final output. The evaluated results are then returned to the user, completing a clear and well-coordinated interaction flow that enhances transparency and user satisfaction.

CHAPTER 6

SOFTWARE ENVIRONMENT

6.1 Software Requirements

Python is a high-level, interpreted programming language known for its simplicity and readability, which makes it a popular choice for beginners as well as experienced developers. Key features of Python include its dynamic typing, automatic memory management, and a rich standard library that supports a wide range of applications from web development to data science and machine learning. Its object-oriented approach and support for multiple programming paradigms allow developers to write clear, maintainable code. Python's extensive ecosystem of third-party packages further enhances its capabilities, enabling rapid development and prototyping across diverse fields.

Installation

First, download the appropriate installer from the official Python website (https://www.python.org/downloads/release/python-376/). For Windows users, run the executable installer and ensure to check the "Add Python to PATH" option during installation; for macOS and Linux, follow the respective package installation commands or use a package manager like Homebrew or apt-get. After installation, verify the setup by running python --version or python3 --version in your terminal or command prompt, which should display "Python 3.12.0." This version-specific installation supports all major functionalities and libraries compatible with Python 3.7.6, making it an excellent foundation for developing robust applications in areas such as data analysis, machine learning, and GUI development.

6.1.1 Python Packages

Core Python Utilities and File Handling Packages: The project makes use of built-in Python libraries such as os, re, and joblib to handle file system operations, text pattern matching, and model persistence. The os module is used to create directories for storing trained models and results, while re supports regular expression–based text cleaning during NLP preprocessing. The joblib library enables efficient saving and loading of trained machine learning models for reuse during prediction.

Numerical Computation and Data Manipulation Packages: NumPy (numpy) and Pandas (pandas) form the backbone of numerical computation and dataset handling. NumPy is used

for efficient array operations, vector stacking, and mathematical computations on GloVe embeddings. Pandas is utilized to load the BBC dataset from CSV files, manage data frames, remove duplicates, and organize textual and numerical features throughout the pipeline.

Natural Language Processing (NLP) Packages: The Natural Language Toolkit (nltk) is used extensively for text preprocessing tasks. It provides tokenization (word_tokenize), stop-word removal (stopwords), part-of-speech tagging (pos_tag), n-gram generation, and lemmatization (WordNetLemmatizer). These NLP operations clean and standardize raw BBC news articles, making them suitable for semantic feature extraction.

Feature Extraction and Linguistic Analysis Packages: GloVe-based feature extraction relies on NumPy for handling dense vector representations. Additionally, linguistic analysis during EDA uses NLTK utilities such as POS tagging and n-gram generation. These packages support the transformation of textual data into fixed-length numerical vectors and help analyze linguistic patterns within the dataset.

Machine Learning and Model Building Packages: The Scikit-learn (sklearn) library is used to implement all supervised classification models in the project. This includes RadiusNeighborsClassifier, DecisionTreeClassifier, GaussianNB, and KNeighborsClassifier. Scikit-learn also provides utilities for train–test splitting, label encoding, and stratified sampling, ensuring consistent and reliable model training.

Model Evaluation and Performance Measurement Packages: Scikit-learn's metrics module is used to evaluate model performance using accuracy, precision, recall, F1-score, confusion matrices, and AUC–ROC curves. The label_binarize function supports multi-class ROC analysis, enabling detailed comparison between different classification models.

Data Visualization Packages: Matplotlib (matplotlib.pyplot) and Seaborn (seaborn) are used for graphical visualization of results and EDA outputs. These packages generate word clouds, bar plots, histograms, confusion matrices, and performance comparison graphs, helping in visual interpretation of model behavior and dataset characteristics.

Text Visualization and Statistical Analysis Packages: The wordcloud library is used to generate word cloud visualizations that highlight frequently occurring terms in BBC news

articles. The collections.Counter class supports frequency analysis of words and bigrams, aiding in exploratory data analysis and feature understanding.

6.2 Hardware Requirements

Python 3.12.0 can run efficiently on most modern systems with minimal hardware requirements. However, meeting the recommended specifications ensures better performance, especially for developers handling large-scale applications or computationally intensive tasks. By ensuring compatibility with hardware and operating system, can leverage the full potential of Python 3.12.0.

Processor (CPU) Requirements: Python 3.12.0 is a lightweight programming language that can run on various processors, making it highly versatile. However, for optimal performance, the following processor specifications are recommended:

- **Minimum Requirement**: 1 GHz single-core processor.
- **Recommended**: Dual-core or quad-core processors with a clock speed of 2 GHz or higher. Using a multi-core processor allows Python applications, particularly those involving multithreading or multiprocessing, to execute more efficiently.

Memory (RAM) Requirements: Python 3.12.0 does not demand excessive memory but requires adequate RAM for smooth performance, particularly for running resource-intensive applications such as data processing, machine learning, or web development.

- **Minimum Requirement**: 512 MB of RAM.
- **Recommended**: 4 GB or higher for general usage. For data-intensive operations, 8 GB or more is advisable.

Insufficient RAM can cause delays or crashes when handling large datasets or executing computationally heavy programs.

Storage Requirements: Python 3.12.0 itself does not occupy significant disk space, but additional storage may be required for Python libraries, modules, and projects.

- **Minimum Requirement**: 200 MB of free disk space for installation.
- **Recommended**: At least 1 GB of free disk space to accommodate libraries and dependencies.

Developers using Python for large-scale projects or data science should allocate more storage to manage virtual environments, datasets, and frameworks like TensorFlow or PyTorch.

Compatibility with Operating Systems: Python 3.12.0 is compatible with most operating systems but requires hardware that supports the respective OS. Below are general requirements for supported operating systems:

- **Windows**: 32-bit and 64-bit systems, Windows 7 or later.
- **macOS**: macOS 10.9 or later.
- **Linux**: Supports a wide range of distributions, including Ubuntu, CentOS, and Fedora.

The hardware specifications for the OS directly impact Python's performance, particularly for modern software development.

CHAPTER 7
FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

This section outlines the functional and non-functional requirements of the proposed fine-grained BBC news topic categorization system, followed by a feasibility (system study) analysis. Requirements analysis defines what the system should do and how well it should perform under real-world conditions. The feasibility study evaluates whether the system is practical to implement from technical, economic, and operational perspectives. Together, these analyses ensure that the proposed solution is realistic, efficient, and capable of meeting user and system expectations.

7.1 Functional Requirements

The proposed fine-grained BBC news topic categorization system is designed to automatically process and classify large volumes of news articles using supervised machine learning techniques. The system accepts BBC news data in CSV format and performs NLP-based preprocessing operations such as text cleaning, tokenization, stop-word removal, and lemmatization to standardize raw textual input. It then extracts semantic features using GloVe word embeddings and converts categorical topic labels into numerical form for model compatibility. The system supports stratified train–test splitting to ensure balanced class distribution during model training. Multiple classification models, including Radius Neighbors Classifier, Decision Tree Classifier, Gaussian Naïve Bayes, and K-Nearest Neighbors, are trained and evaluated using standard performance metrics such as accuracy, precision, recall, F1-score, confusion matrix, and AUC–ROC curves. Additionally, the system allows users to perform both single-text and batch-wise predictions on unseen news articles, while enabling trained models to be saved and reloaded for future use, ensuring automation, consistency, and efficient news categorization.

- The system shall load and process BBC news articles from CSV files.
- The system shall perform NLP preprocessing including text cleaning, tokenization, stop-word removal, and lemmatization.
- The system shall extract semantic features using GloVe word embeddings.
- The system shall split the dataset into training and testing sets using stratified sampling.

- The system shall train supervised classifiers including Radius Neighbors Classifier, Decision Tree Classifier, Gaussian Naïve Bayes, and K-Nearest Neighbors (KNN).
- The system shall evaluate models using accuracy, precision, recall, F1-score, confusion matrix, and AUC–ROC metrics.
- The system shall compare the performance of all implemented models.
- The system shall support single news article prediction.
- The system shall support batch-wise prediction using CSV input files.
- The system shall store and reload trained models for future predictions.

7.2 Non-Functional Requirements

The proposed fine-grained BBC news topic categorization system is required to deliver accurate, reliable, and efficient performance under varying data conditions. The system should ensure high classification accuracy and consistent predictions across different news categories while handling large volumes of textual data without significant performance degradation. It must be scalable to accommodate future dataset expansions and additional topic categories. Reproducibility of results is ensured through fixed random states and standardized preprocessing procedures. The system should be robust enough to handle missing or noisy input data gracefully without causing execution failures. Furthermore, it should be portable across different operating systems that support Python and associated libraries, maintain acceptable execution time for training and prediction, and provide ease of use for users with minimal technical expertise, thereby ensuring long-term usability and maintainability.

- The system should provide accurate and consistent topic predictions.
- The system should handle large volumes of news articles efficiently.
- The system should ensure reproducibility of results using fixed random states.
- The system should be scalable to support additional datasets and categories.
- The system should be reliable and handle missing or invalid input gracefully.
- The system should be user-friendly and easy to operate for both training and prediction.
- The system should maintain acceptable execution time for model evaluation and prediction.

- The system should be portable across different operating systems with Python support.

7.3 System Study

Feasibility Type	Description
Technical Feasibility	The system is technically feasible as it uses well-established Python libraries such as NumPy, Pandas, NLTK, and Scikit-learn. GloVe embeddings and supervised classifiers are readily available and compatible with the system architecture.
Economic Feasibility	The project is cost-effective since it relies on open-source tools and pre-trained embeddings. No additional hardware or licensed software is required for implementation.
Operational Feasibility	The system is easy to operate and does not require extensive user training. Automated preprocessing, model training, and prediction reduce manual effort and operational complexity.
Schedule Feasibility	The project can be completed within the planned timeframe due to the modular design and availability of datasets, libraries, and development tools.
Scalability Feasibility	The system can be extended to larger datasets, additional news categories, or advanced models without major architectural changes.

CHAPTER 8

SOURCE CODE

```
# Fine Grained Topic Categorization of BBC News Articles using Supervised Document Classification Models

## import libraries

import os

import re

import joblib

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import nltk


from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.naive_bayes import GaussianNB

from sklearn.neighbors import KNeighborsClassifier, RadiusNeighborsClassifier

from sklearn.metrics import (

    accuracy_score,

    classification_report,

    precision_score,

    recall_score,

    f1_score,

    confusion_matrix,

    roc_curve,
```

```
    auc
)
from sklearn.preprocessing import label_binarize, LabelEncoder

from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

nltk.download('punkt', quiet=True)
nltk.download('stopwords', quiet=True)
nltk.download('wordnet', quiet=True)
nltk.download('averaged_perceptron_tagger', quiet=True)
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger_eng')

sw = set(stopwords.words('english'))
lm = WordNetLemmatizer()

## Dataset Loading
df=pd.read_csv('Dataset/bbc_data.csv')
df.drop_duplicates()
## Data preprocessing
def preprocess_text_df(df, text_col='data', label_col='labels'):
```

```
    def clean(t):
        t = str(t).lower()
        t = re.sub(r'[^a-z0-9\s]', ' ', t)
        tokens = word_tokenize(t)
        tokens = [lm.lemmatize(w) for w in tokens if w not in sw]
        return " ".join(tokens)

    df['clean_text'] = df[text_col].apply(clean)
    le = LabelEncoder()
    df['label_encoded'] = le.fit_transform(df[label_col])

    label_mapping = dict(zip(le.classes_, le.transform(le.classes_)))
    print(label_mapping)

    return df, le



df_processed, label_encoder = preprocess_text_df(df, text_col='data', label_col='labels')

## Feature Extraction

def load_glove_100(glove_path):
    embeddings = {}
```

```
    with open(glove_path, encoding="utf8") as f:
        for line in f:
            parts = line.split()
            word = parts[0]
            vec = np.asarray(parts[1:], dtype="float32")
            embeddings[word] = vec
    return embeddings

def text_to_glove_vector(text, embeddings, dim=100):
    if not isinstance(text, str):
        return np.zeros(dim).tolist()
    words = text.split()
    vectors = [embeddings[w] for w in words if w in embeddings]
    if len(vectors) == 0:
        return np.zeros(dim).tolist()
    return np.mean(vectors, axis=0).tolist()

def df_to_glove_embeddings(df, text_col, embeddings, dim=100):
    df[text_col] = df[text_col].fillna("")
    df['glove_vec'] = df[text_col].apply(lambda x: text_to_glove_vector(x, embeddings, dim))
    return df

embeddings = load_glove_100("glove.6B.100d.txt")
df = df_to_glove_embeddings(df, "clean_text", embeddings)
```

```
X = np.vstack(df['glove_vec'].values)
y = df['label_encoded'].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
## EDA (Exploratory Data Analysis)
import matplotlib.pyplot as plt
import seaborn as sns
from collections import Counter
from wordcloud import WordCloud
from nltk.tokenize import word_tokenize
from nltk import pos_tag
from nltk.util import ngrams

def eda_nlp_analysis(X_text, num_words=100, top_n_words=20):
    print("Generating NLP EDA Visualizations...")

    all_tokens = [word for doc in X_text for word in word_tokenize(doc)]

    word_freq = Counter(all_tokens)
    wc = WordCloud(
        width=800,
        height=400,
```

```
    max_words=num_words,
    background_color='white'
).generate_from_frequencies(word_freq)

plt.figure(figsize=(10, 5))
plt.imshow(wc, interpolation='bilinear')
plt.axis('off')
plt.show()

common_words = word_freq.most_common(top_n_words)
words, counts = zip(*common_words)

plt.figure(figsize=(10, 5))
sns.barplot(x=list(counts), y=list(words))
plt.show()

doc_lengths = [len(word_tokenize(doc)) for doc in X_text]

plt.figure(figsize=(10, 5))
plt.hist(doc_lengths, bins=20)
plt.xlabel("Document Length")
plt.ylabel("Frequency")
plt.title("Distribution of Document Lengths")
plt.show()
```

```
    all_pos = [tag for _, tag in pos_tag(all_tokens)]
    pos_counts = Counter(all_pos).most_common()
    pos_tags, pos_freqs = zip(*pos_counts)

    plt.figure(figsize=(10, 5))
    sns.barplot(x=list(pos_tags), y=list(pos_freqs))
    plt.xticks(rotation=45)
    plt.show()

    bigrams = list(ngrams(all_tokens, 2))
    bigram_freq = Counter(bigrams).most_common(top_n_words)
    bigram_labels = [' '.join(b) for b, _ in bigram_freq]
    bigram_counts = [count for _, count in bigram_freq]

    plt.figure(figsize=(10, 5))
    sns.barplot(x=bigram_counts, y=bigram_labels)
    plt.show()


X_text = df["clean_text"].astype(str).tolist()
eda_nlp_analysis(X_text, num_words=100, top_n_words=20)


os.makedirs("results", exist_ok=True)
```

```
label_map = {
    0: "business",
    1: "entertainment",
    2: "politics",
    3: "sport",
    4: "tech"
}

c = list(label_map.values())

num_classes = len(np.unique(y_test))
y_test_bin = label_binarize(y_test, classes=list(range(num_classes)))


def plot_and_save_auc_roc(model, X_test, y_test, model_name):
    if hasattr(model, "predict_proba"):
        y_score = model.predict_proba(X_test)
    else:
        y_score = label_binarize(
            model.predict(X_test),
            classes=list(range(num_classes))
        )

    plt.figure()
```

```
    for i in range(num_classes):
        fpr, tpr, _ = roc_curve(y_test_bin[:, i], y_score[:, i])
        roc_auc = auc(fpr, tpr)
        plt.plot(fpr, tpr, label=f"{c[i]} (AUC={roc_auc:.3f})")

    plt.plot([0, 1], [0, 1], linestyle="--")
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    plt.title(f"AUC-ROC Curve - {model_name}")
    plt.legend()

    path = f"results/{model_name}_auc_roc.png"
    plt.savefig(path, dpi=300)
    plt.show()
    plt.close()

    print("Saved:", path)


def plot_and_save_confusion_matrix(y_true, y_pred, model_name):
    cm = confusion_matrix(y_true, y_pred)

    plt.figure(figsize=(7, 6))
    plt.imshow(cm, cmap="Blues")
```

```
    plt.title(f"Confusion Matrix - {model_name}")
    plt.xlabel("Predicted Label")
    plt.ylabel("True Label")
    plt.colorbar()

    plt.xticks(range(len(c)), c)
    plt.yticks(range(len(c)), c)

    thresh = cm.max() / 2
    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            plt.text(
                j, i, format(cm[i, j], "d"),
                ha="center", va="center",
                color="white" if cm[i, j] > thresh else "black"
            )

    plt.tight_layout()
    plt.savefig(f"results/{model_name}_confusion_matrix.png", dpi=300)
    plt.show()
    plt.close()

## Model Training and Building
```

```
os.makedirs("model", exist_ok=True)

rnc_model = RadiusNeighborsClassifier(
    radius=5.0,
    weights="uniform",
    metric="euclidean",
    outlier_label=0
)

rnc_model.fit(X_train, y_train)

joblib.dump(rnc_model, "model/radius_neighbors.pkl")

y_train_pred = rnc_model.predict(X_train)
y_test_pred = rnc_model.predict(X_test)

test_acc = accuracy_score(y_test, y_test_pred)
test_precision = precision_score(y_test, y_test_pred, average='weighted', zero_division=0)
test_recall = recall_score(y_test, y_test_pred, average='weighted', zero_division=0)
test_f1 = f1_score(y_test, y_test_pred, average='weighted', zero_division=0)

print("ACCURACY:", round(test_acc * 100, 2), "%")
print("PRECISION:", round(test_precision, 2))
print("RECALL:", round(test_recall, 2))
```

```
print("F1 SCORE:", round(test_f1, 2))
print(classification_report(y_test, y_test_pred, zero_division=0))

plot_and_save_confusion_matrix(y_test, y_test_pred, "Radius_Neighbors")

plot_and_save_auc_roc(rnc_model, X_test, y_test, "Radius_Neighbors")

os.makedirs("model", exist_ok=True)

dt_model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=None,
    random_state=42
)

dt_model.fit(X_train, y_train)

joblib.dump(dt_model, "model/decision_tree.pkl")

y_train_pred = dt_model.predict(X_train)
y_test_pred = dt_model.predict(X_test)

test_acc = accuracy_score(y_test, y_test_pred)
test_precision = precision_score(y_test, y_test_pred, average='weighted', zero_division=0)
```

```
test_recall = recall_score(y_test, y_test_pred, average='weighted', zero_division=0)
test_f1 = f1_score(y_test, y_test_pred, average='weighted', zero_division=0)

print("ACCURACY:", round(test_acc * 100, 2), "%")
print("PRECISION:", round(test_precision, 2))
print("RECALL:", round(test_recall, 2))
print("F1 SCORE:", round(test_f1, 2))
print(classification_report(y_test, y_test_pred, zero_division=0))

dt_model = joblib.load("model/decision_tree.pkl")

y_pred_dt = dt_model.predict(X_test)

plot_and_save_confusion_matrix(y_test, y_pred_dt, "Decision_Tree")

plot_and_save_auc_roc(dt_model, X_test, y_test, "Decision_Tree")



os.makedirs("model", exist_ok=True)

naive_bayes_model = GaussianNB()
naive_bayes_model.fit(X_train, y_train)
```

```
joblib.dump(naive_bayes_model, "model/naive_bayes.pkl")


y_train_pred = naive_bayes_model.predict(X_train)

y_test_pred = naive_bayes_model.predict(X_test)


test_acc = accuracy_score(y_test, y_test_pred)

test_precision = precision_score(y_test, y_test_pred, average='weighted', zero_division=0)

test_recall = recall_score(y_test, y_test_pred, average='weighted', zero_division=0)

test_f1 = f1_score(y_test, y_test_pred, average='weighted', zero_division=0)


print("ACCURACY:", round(test_acc * 100, 2), "%")

print("PRECISION:", round(test_precision, 2))

print("RECALL:", round(test_recall, 2))

print("F1 SCORE:", round(test_f1, 2))

print(classification_report(y_test, y_test_pred, zero_division=0))


nb_model = joblib.load("model/naive_bayes.pkl")

y_pred_nb = nb_model.predict(X_test)


plot_and_save_confusion_matrix(y_test, y_pred_nb, "Naive_Bayes")

plot_and_save_auc_roc(nb_model, X_test, y_test, "Naive_Bayes")



os.makedirs("model", exist_ok=True)
```

```
knn_model = KNeighborsClassifier(
    n_neighbors=1,
    metric="euclidean",
    weights="uniform"
)

knn_model.fit(X_train, y_train)

joblib.dump(knn_model, "model/knn_classifier.pkl")

y_train_pred = knn_model.predict(X_train)
y_test_pred = knn_model.predict(X_test)

test_acc = accuracy_score(y_test, y_test_pred)
test_precision = precision_score(y_test, y_test_pred, average='weighted', zero_division=0)
test_recall = recall_score(y_test, y_test_pred, average='weighted', zero_division=0)
test_f1 = f1_score(y_test, y_test_pred, average='weighted', zero_division=0)

print("ACCURACY:", round(test_acc * 100, 2), "%")
print("PRECISION:", round(test_precision, 2))
print("RECALL:", round(test_recall, 2))
print("F1 SCORE:", round(test_f1, 2))
print(classification_report(y_test, y_test_pred, zero_division=0))
```

```
plot_and_save_confusion_matrix(y_test, y_test_pred, "KNN_Low_Accuracy")

plot_and_save_auc_roc(knn_model, X_test, y_test, "KNN_Low_Accuracy")



import os

import joblib

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt


from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score


os.makedirs("results", exist_ok=True)


models = {

    "Radius Neighbors": joblib.load("model/radius_neighbors.pkl"),

    "Decision Tree": joblib.load("model/decision_tree.pkl"),

    "Naive Bayes": joblib.load("model/naive_bayes.pkl"),

    "KNN": joblib.load("model/knn_classifier.pkl"),

}


results = []


for name, model in models.items():
```

```
    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, average="weighted", zero_division=0)
    rec = recall_score(y_test, y_pred, average="weighted", zero_division=0)
    f1 = f1_score(y_test, y_pred, average="weighted", zero_division=0)

    results.append([name, acc * 100, prec * 100, rec * 100, f1 * 100])

metrics_df = pd.DataFrame(
    results,
    columns=["Algorithm", "Accuracy", "Precision", "Recall", "F1 Score"]
)



metrics = ["Accuracy", "Precision", "Recall", "F1 Score"]

for metric in metrics:
    plt.figure(figsize=(8, 5))
    plt.bar(metrics_df["Algorithm"], metrics_df[metric])
    plt.title(f"{metric} Comparison")
    plt.ylabel(f"{metric} (%)")
    plt.xlabel("Algorithm")
```

```
    plt.xticks(rotation=30)
    plt.grid(axis="y")
    plt.tight_layout()
    plt.savefig(f"results/{metric.lower().replace(' ', '_')}_comparison.png")
    plt.show()

print("\nAll plots saved in the 'results/' directory")

# Testing with Prediction Single prediction and Batchwise Predictions
### Preprocessing for Test
model = joblib.load("model/knn_classifier.pkl")

label_map = {
    0: "business",
    1: "entertainment",
    2: "politics",
    3: "sport",
    4: "tech"
}

def load_glove_100(glove_path):
    embeddings = {}
    with open(glove_path, encoding="utf8") as f:
        for line in f:
```

```
            parts = line.split()
            embeddings[parts[0]] = np.asarray(parts[1:], dtype="float32")
    return embeddings

embeddings = load_glove_100("glove.6B.100d.txt")

sw = set(stopwords.words('english'))
lm = WordNetLemmatizer()

def preprocess_text(text):
    text = str(text).lower()
    text = re.sub(r'[^a-z0-9\s]', ' ', text)
    text = re.sub(r'\s+', ' ', text).strip()
    tokens = word_tokenize(text)
    tokens = [lm.lemmatize(w) for w in tokens if w not in sw]
    return " ".join(tokens)

def text_to_vector(text, dim=100):
    words = text.split()
    vecs = [embeddings[w] for w in words if w in embeddings]
    if len(vecs) == 0:
        return np.zeros(dim)
    return np.mean(vecs, axis=0)
```

```
def predict_from_text(text):
    cleaned = preprocess_text(text)
    vec = text_to_vector(cleaned)
    vec = np.array(vec).reshape(1, -1)
    pred = model.predict(vec)[0]
    return label_map[int(pred)]


def predict_batch_from_csv(csv_path, text_column="data"):
    df = pd.read_csv(csv_path)
    df[text_column] = df[text_column].fillna("")


    print("\n===== ALL BATCH PREDICTIONS =====\n")


    for text in df[text_column]:
        category = predict_from_text(text)
        print(text)
        print("➜ Predicted Category:", category)
        print("-" * 100)
### Single Prediction
sample_text = """Rocker Doherty in on-stage fight  Rock singer Pete Doherty has been involved in a fight with his bands guitarist at their biggest gig to date.  Babyshambles played for 5,000 fans at Londons Brixton Academy on Tuesday. The former Libertines singer traded blows with guitarist Patrick Walden. They were separated and bundled off stage before returning to end the set. The show was earlier held up due to audience overcrowding. On Monday Doherty faced blackmail and robbery charges in court, which he denies. He is out on xc2xa350,000 bail and the judge agreed to extend his 2200 GMT curfew deadline by two
```

hours so he could play the Brixton gig. Babyshambles, which he formed after his acrimonious departure from the Libertines, played a warm-up show at The Garage, north London, on Monday. On Tuesday, Doherty and his three bandmates were introduced to the crowd by Mick Jones, the former Clash guitarist who produced the Libertines second album. Babyshambles took the stage to a frenzied reception at 2200 GMT, launching into their last single, Killamangiro, which reached number eight in December. But the group had to stop during the next song to persuade fans not to push forward and allow security guards to pull people out of the crush. Doherty appealed to fans to calm down, saying: "Theres a few people getting hurt down the front, youve got to move back." The music resumed minutes later but after several more songs, the singer appeared to accidentally disconnect Waldens guitar, leading the pair to trade kicks and punches. Bouncers stepped in and the band left the stage, but returned after five minutes and finished their 50-minute performance with no further trouble. Doherty, 25, had to be home by midnight to observe the curfew, which is one of the conditions of his bail. On Monday, the judge agreed to allow him to perform on condition that requests for changes to his bail terms were "not too regular an occurrence". The singer was arrested with fellow musician Alan Wass on 2 February after an alleged dispute with documentary-maker Max Carlish at a London hotel. They are next due in court on 18 April.

"""

```
print("Predicted Category:", predict_from_text(sample_text))
```

Batwise Prediction

```
predict_batch_from_csv("Dataset/test.csv", text_column="data")
```

CHAPTER 9

RESULTS AND DISCUSSION

9.1 Implementation description

Library Initialization and Environment Setup: The implementation begins by importing core Python, NLP, machine learning, and visualization libraries such as NumPy, Pandas, NLTK, and Scikit-learn. These libraries establish the computational environment required for text processing, feature extraction, model training, evaluation, and result visualization. Required NLTK resources are downloaded dynamically to support tokenization, lemmatization, and linguistic analysis.

Key commands involved: import nltk, from sklearn…, nltk.download()

Dataset Loading and Initial Cleaning: The BBC news dataset is loaded from a CSV file using Pandas, and duplicate records are removed to avoid biased learning. This dataset forms the primary input and contains raw news text along with corresponding topic labels. The cleaned dataset is passed forward for preprocessing and feature generation.

Key commands involved: pd.read_csv(), df.drop_duplicates()

NLP Preprocessing Pipeline Construction: A preprocessing function is defined to clean raw news articles by converting text to lowercase, removing special characters, tokenizing words, eliminating stopwords, and applying lemmatization. This function is applied to the entire dataset, creating a new clean_text column that contains normalized textual content suitable for semantic analysis.

Key commands involved: re.sub(), word_tokenize(), WordNetLemmatizer(), df.apply()

Label Encoding and Class Mapping: The categorical topic labels are converted into numerical form using label encoding. This transformation enables compatibility with supervised learning algorithms while preserving class identity through a printed label-to-index mapping.

Key commands involved: LabelEncoder(), fit_transform()

GloVe Embedding Model Loading: Pre-trained GloVe vectors (100-dimensional) are loaded into memory as a dictionary mapping words to vectors. This step establishes a semantic lookup table that allows conversion of words into dense numerical representations.

Key commands involved: open(glove.6B.100d.txt), np.asarray()

Document-Level Feature Vector Generation: Each cleaned news article is transformed into a fixed-length vector by averaging the GloVe vectors of all valid words in the text. If no valid words are found, a zero vector is assigned to maintain dimensional consistency. All document vectors are stacked to form the final feature matrix.

Key commands involved: np.mean(), np.vstack()

Train–Test Split with Stratification: The feature matrix and encoded labels are split into training and testing sets using an 80:20 ratio. Stratified sampling is applied to preserve class distribution across both sets, ensuring fair and unbiased model evaluation.

Key commands involved: train_test_split(stratify=y)

Exploratory Data Analysis (EDA) Integration: Linguistic and statistical patterns are explored using word clouds, word frequency plots, document length histograms, POS tag distributions, and bigram analysis. These visualizations validate preprocessing quality and provide insight into dataset structure.

Key commands involved: WordCloud(), Counter(), pos_tag(), ngrams()

Model Training: Radius Neighbors Classifier: The Radius Neighbors Classifier is trained using Euclidean distance and a fixed radius. It learns topic assignments based on local semantic neighborhoods and handles outliers using a predefined label. The trained model is saved for reuse.

Key commands involved: RadiusNeighborsClassifier(), fit(), joblib.dump()

Model Training: Decision Tree Classifier: A Decision Tree model is trained using the Gini impurity criterion to learn hierarchical decision rules from GloVe features. This model captures non-linear feature relationships and provides interpretable classification logic.

Key commands involved: DecisionTreeClassifier(), fit()

Model Training: Gaussian Naïve Bayes Classifier: The Gaussian Naïve Bayes model learns probabilistic distributions of features for each topic class. It computes class-conditional likelihoods assuming Gaussian feature distributions and is optimized for fast execution.

Key commands involved: GaussianNB(), fit()

Proposed Model Training: K-Nearest Neighbors (KNN): The KNN classifier is trained using Euclidean distance with k=1, classifying news articles based on the closest semantic neighbor. This instance-based approach directly leverages GloVe similarity for fine-grained topic prediction.

Key commands involved: KNeighborsClassifier(), fit()

Model Evaluation and Visualization: All models are evaluated using accuracy, precision, recall, F1-score, confusion matrices, and multi-class AUC-ROC curves. Evaluation results are visualized and saved to the results directory for comparative analysis.

Key commands involved: accuracy_score(), confusion_matrix(), roc_curve(), plt.savefig()

Model Comparison and Performance Analysis: Trained models are reloaded and evaluated on the same test dataset to ensure fair comparison. Performance metrics are stored in a DataFrame and visualized using bar charts for clear comparison across algorithms.

Key commands involved: joblib.load(), pd.DataFrame(), plt.bar()

Single Text Prediction Workflow: A new news article is preprocessed, converted into a GloVe vector, and passed to the trained KNN model for prediction. The predicted numerical label is mapped back to a human-readable topic category.

Key commands involved: predict(), reshape(), label_map

Batch-Wise Prediction Workflow: Multiple unseen news articles are loaded from a CSV file and classified one by one using the same preprocessing and feature extraction pipeline. Predicted categories are printed sequentially, enabling large-scale automated classification.

Key commands involved: pd.read_csv(), loop-based predict_from_text()

9.2 Dataset description

The dataset used in this project is derived from the BBC News text corpus and focuses specifically on the Entertainment category. It consists of news articles written in natural language, covering a wide range of entertainment-related topics such as music, films, celebrities, awards, television, festivals, and cultural events. Each record in the dataset contains two main fields: a data column that holds the full news article text (including headlines and detailed content) and a labels column that represents the class category. In this implementation, all samples are labeled as *entertainment*, making the dataset suitable for studying fine-grained text representation, NLP preprocessing, semantic feature extraction using GloVe vectors, and supervised document classification workflows. The dataset is rich in vocabulary, varied sentence structures, and real-world linguistic patterns, which makes it ideal for evaluating the performance of multiple machine learning classifiers in an NLP-based text classification system.

Attribute Name	**Description**
Dataset Source	BBC News Dataset
Domain	Natural Language Processing (Text Classification)
Category	Entertainment
Data Type	Textual (Unstructured Data)
Input Feature	News article text (headlines + content)
Target Label	entertainment
Language	English
Content Coverage	Music, movies, celebrities, awards, TV shows, festivals, culture
Usage Purpose	NLP preprocessing, GloVe-based feature extraction, supervised classification
Learning Type	Supervised Learning
Dataset Nature	Real-world news articles

9.3 Results description

The figure 9.1 illustrates a sample view of the BBC news dataset used in the proposed fine-grained topic categorization system. It shows a tabular representation with two primary columns: data, which contains the raw textual content of BBC news articles, and labels, which represent the corresponding topic categories assigned to each article. The sample records demonstrate that the dataset includes diverse news headlines and descriptions spanning multiple domains such as entertainment and technology. Each row corresponds to a unique news article, highlighting real-world language usage, varied sentence structures, and domain-specific vocabulary. This snapshot confirms the structured format of the dataset, where unstructured text is systematically paired with its class label, making it suitable for NLP preprocessing, semantic feature extraction, and supervised document classification experiments.

	data	labels
0	Musicians to tackle US red tape Musicians gro...	entertainment
1	U2s desire to be number one U2, who have won ...	entertainment
2	Rocker Doherty in on-stage fight Rock singer ...	entertainment
3	Snicket tops US box office chart The film ada...	entertainment
4	Oceans Twelve raids box office Oceans Twelve,...	entertainment
...	...	...
2220	Warning over Windows Word files Writing a Mic...	tech
2221	Fast lifts rise into record books Two high-sp...	tech
2222	Nintendo adds media playing to DS Nintendo is...	tech
2223	Fast moving phone viruses appear Security fir...	tech
2224	Hacker threat to Apples iTunes Users of Apple...	tech

2126 rows × 2 columns

Fig. 9.1: Sample records from the dataset.

The figure 9.2(a) presents a word cloud visualization highlighting the most frequent terms occurring in the BBC news dataset after NLP preprocessing. In this representation, words such as “said,” “year,” “people,” “world,” “one,” and “new” appear more prominently, indicating their high frequency across multiple news articles. The varying font sizes reflect the relative occurrence of each term, allowing quick identification of dominant vocabulary within the dataset. This visualization provides an intuitive overview of the dataset’s thematic emphasis and confirms the presence of common news-reporting terms, thereby validating the

effectiveness of text preprocessing and offering valuable insight into the linguistic patterns present in the corpus.

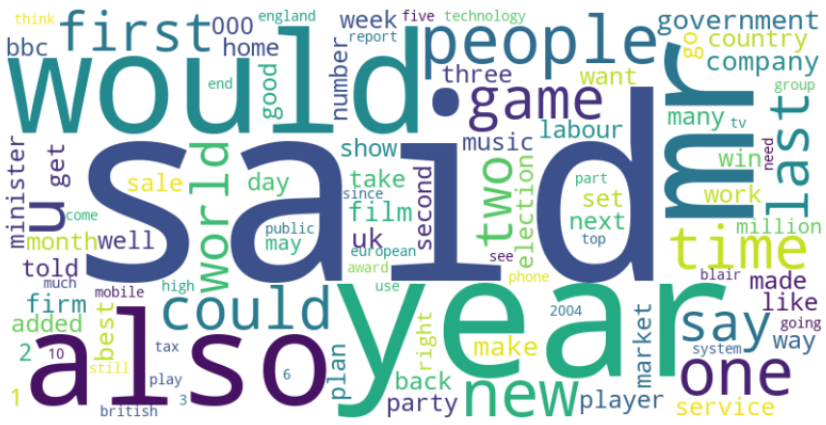


Fig. 9.2 (a): Word Cloud of Most Frequent Terms in Dataset.

The figure 9.2(b) illustrates a bar chart representing the top 20 most frequent words in the BBC news dataset after preprocessing. The horizontal bars show the frequency distribution of commonly occurring terms such as "said," "year," "mr," "would," "also," "people," and "new," which are typical of news reporting language. The length of each bar corresponds to the number of times the word appears across all articles, clearly highlighting dominant terms within the corpus. This visualization provides a quantitative understanding of word usage patterns, complements the word cloud representation, and helps validate the effectiveness of NLP preprocessing by emphasizing meaningful and recurrent vocabulary present in the dataset.

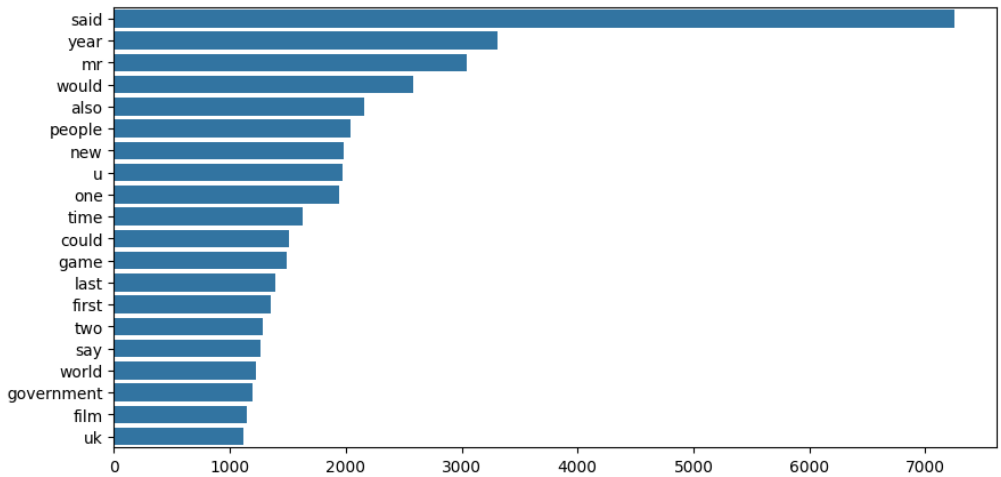


Fig. 9.2 (b): Top 20 Most Frequent Words in the Dataset

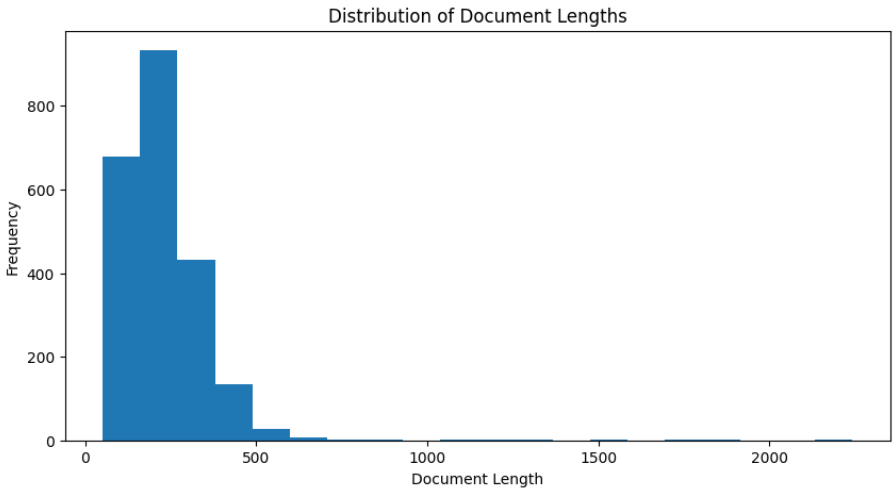


Fig. 9.2 (c): Distribution of Document Lengths in e learning Dataset.

The figure 9.2(c) shows a histogram illustrating the distribution of document lengths in the dataset based on the number of tokens per article after preprocessing. It can be observed that the majority of documents fall within a shorter length range, indicating that most BBC news articles consist of a moderate number of words, while a smaller number of documents extend to much larger lengths. This right-skewed distribution highlights variability in article size, reflecting differences between brief news reports and more detailed articles. The visualization helps in understanding the textual structure of the dataset and confirms that the documents are suitable for vector-based representation and supervised classification without excessive imbalance in text length.

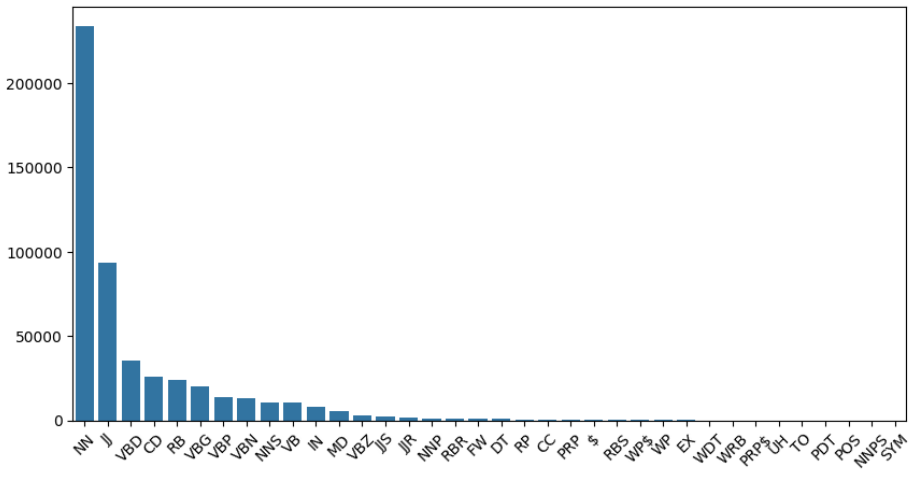


Fig. 9.2 (d): Part-of-Speech (POS) Tag Frequency Distribution.

The figure 9.2(d) presents the Part-of-Speech (POS) tag frequency distribution for the BBC news dataset, showing how different grammatical categories are used across the corpus. Noun-related tags such as NN and NNS dominate the distribution, indicating a strong presence of entities, subjects, and objects typical of news reporting. Verb forms like VBD, VBP, and VBZ also appear frequently, reflecting the action-oriented nature of news articles. The relatively lower frequencies of adjectives, adverbs, and other grammatical tags highlight the informative and factual writing style of the dataset. This analysis provides valuable insight into the linguistic structure of the text and supports effective feature representation for supervised document classification.

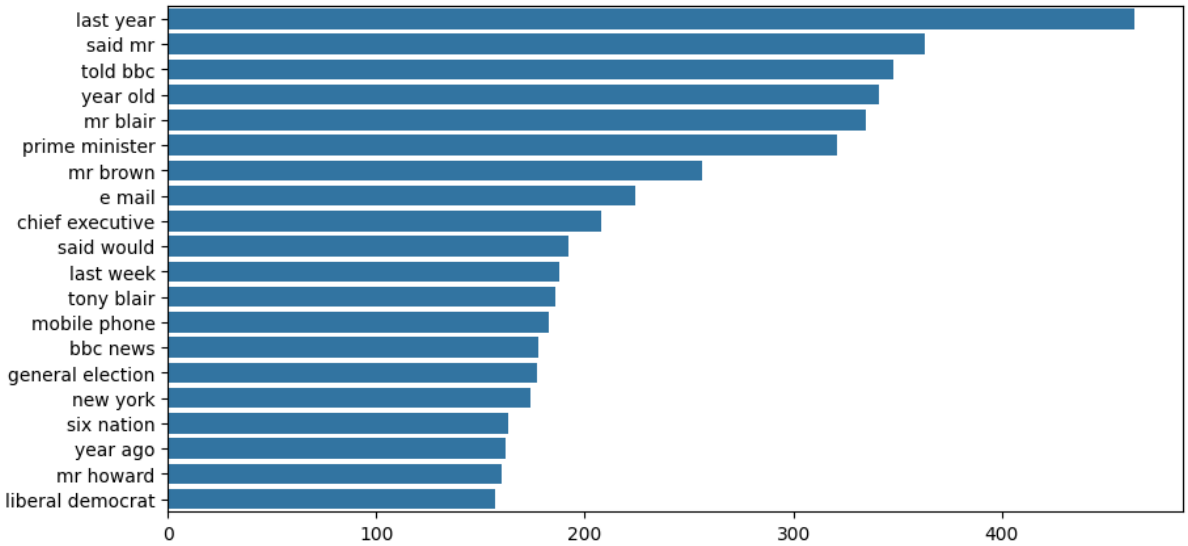
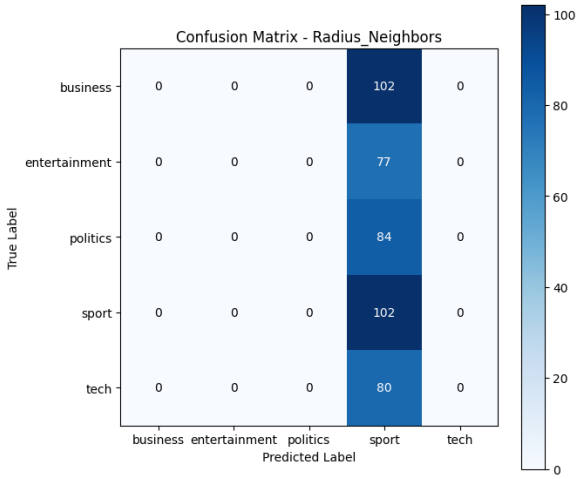
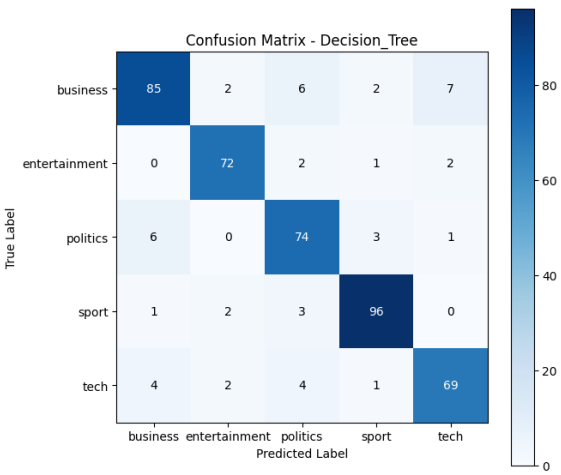


Fig. 9.2 (e): Top 20 Most Frequent Bigrams in the Dataset

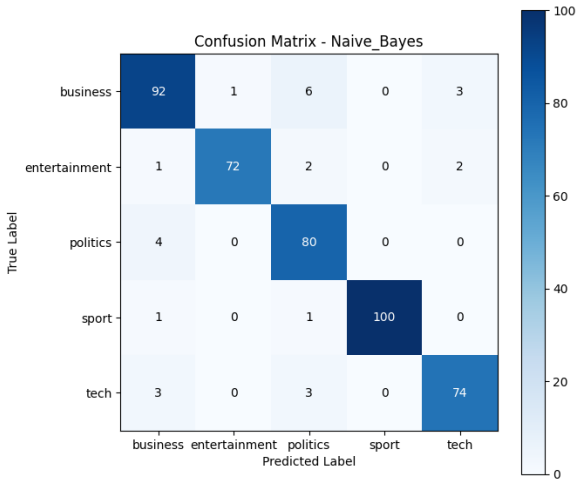
The figure 9.2(e) illustrates the top 20 most frequent bigrams present in the BBC news dataset after preprocessing. Each bar represents a commonly occurring pair of consecutive words such as “last year,” “said mr,” “told bbc,” “prime minister,” and “bbc news,” which are characteristic of journalistic writing and reporting style. The frequency distribution highlights repeated phrase patterns that convey temporal references, quotations, and institutional mentions commonly found in news articles. This bigram analysis provides deeper insight into contextual word relationships beyond individual terms and helps in understanding how meaningful phrases contribute to fine-grained topic categorization.



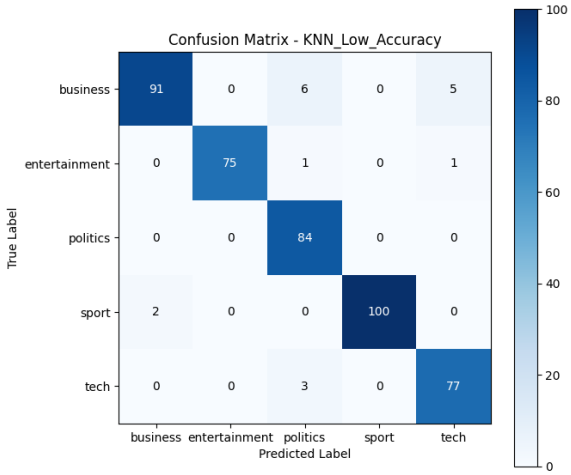
(a)



(b)



(c)



(d)

Fig. 9.3: Confusion matrix analysis of classification models. (a) Radius Neighbors classifier. (b) Decision Tree classifier. (c) Naive bayes classifier. (d) KNN Classifier.

The figure 9.3 (a) presents the confusion matrix obtained for the Radius Neighbors Classifier applied to the BBC news topic categorization task. It clearly shows that the classifier predicts almost all test samples as the sport category, regardless of their true labels such as business, entertainment, politics, or technology. This behavior results in high misclassification rates for non-sport categories, indicating poor class discrimination and severe prediction bias. The absence of correct predictions along the diagonal for most classes highlights the model's inability to form meaningful decision boundaries in the high-dimensional GloVe feature space. Overall, the confusion matrix demonstrates that the Radius Neighbors Classifier

performs inadequately for fine-grained news classification, primarily due to ineffective radius-based neighbor selection and imbalance in semantic clustering.

The figure 9.3 (b) shows the confusion matrix for the Decision Tree Classifier used in the BBC news topic categorization task. The diagonal elements indicate that the model correctly classifies a substantial number of articles across all categories, particularly achieving strong performance for the sport**,** business**,** politics, and entertainmen**t** classes. However, some misclassifications are observed between semantically related categories such as business and politics or technology and business, reflecting overlapping content in news articles. Overall, the confusion matrix demonstrates that the Decision Tree model performs reasonably well with balanced predictions across classes, though its accuracy is limited by overfitting and sensitivity to subtle semantic variations in the GloVe-based feature space.

The figure 9.3 (c) illustrates the confusion matrix obtained for the Gaussian Naïve Bayes classifier in the BBC news topic categorization task. The strong diagonal values indicate that the model correctly classifies a large proportion of articles across all categories, with particularly high accuracy for the sport**,** politics, and business classes. Only a small number of misclassifications occur between closely related categories such as business and politics or technology and business, which is expected due to overlapping terminology in news content. Overall, the confusion matrix demonstrates that Gaussian Naïve Bayes provides stable and well-balanced predictions, though its performance is influenced by the independence and distribution assumptions applied to the GloVe-based feature representations.

The figure 9.3 (d) presents the confusion matrix for the K-Nearest Neighbors (KNN) classifier applied to the BBC news topic categorization task. The diagonal elements show that the classifier correctly predicts a high number of instances for all categories, particularly achieving perfect or near-perfect classification for the sport and politics classes. Minor misclassifications are observed between closely related categories such as business and politics or business and technology, reflecting semantic overlap in news content. Overall, the confusion matrix indicates that the KNN model provides accurate and well-balanced predictions across categories, demonstrating its effectiveness in leveraging GloVe-based semantic similarity for fine-grained news classification.

9.4 Comparative Analysis

Table 2: Performance comparison of classification models

Model	Accuracy	Precision	Recall	F1-Score
RNC	22.92	0.05	0.23	0.09
DTC	88.99	0.89	0.89	0.89
NBC	93.93	0.94	0.94	0.94
KNN	95.96	0.96	0.96	0.96

The table presents a performance comparison of the implemented classification models—Radius Neighbors Classifier (RNC), Decision Tree Classifier (DTC), Naïve Bayes Classifier (NBC), and K-Nearest Neighbors (KNN)—based on accuracy, precision, recall, and F1-score. The Radius Neighbors Classifier shows very poor performance with low accuracy and weak precision and F1-score, indicating its inability to effectively discriminate between news categories. The Decision Tree Classifier achieves strong and balanced performance across all metrics, reflecting its capability to learn hierarchical decision rules from the data. The Naïve Bayes Classifier further improves performance, demonstrating high accuracy and consistent metric values due to its probabilistic modeling. Notably, the KNN classifier outperforms all other models with the highest accuracy, precision, recall, and F1-score, highlighting its effectiveness in leveraging GloVe-based semantic similarity for fine-grained BBC news topic categorization.

The figure 9.4 illustrates a detailed accuracy comparison among the four classification models used in the BBC news topic categorization system. It clearly shows that the Radius Neighbors Classifier performs poorly, achieving very low accuracy compared to the other models, which indicates its inability to effectively handle the semantic structure of the dataset. The Decision Tree model demonstrates strong performance with a high accuracy level, while the Naïve Bayes classifier further improves upon this result, reflecting its effective probabilistic learning. The KNN classifier achieves the highest accuracy among all models, highlighting its superior capability to exploit GloVe-based semantic similarity for fine-grained news classification. Overall, the figure visually emphasizes the progressive improvement in model performance and validates KNN as the most effective approach in the proposed system.

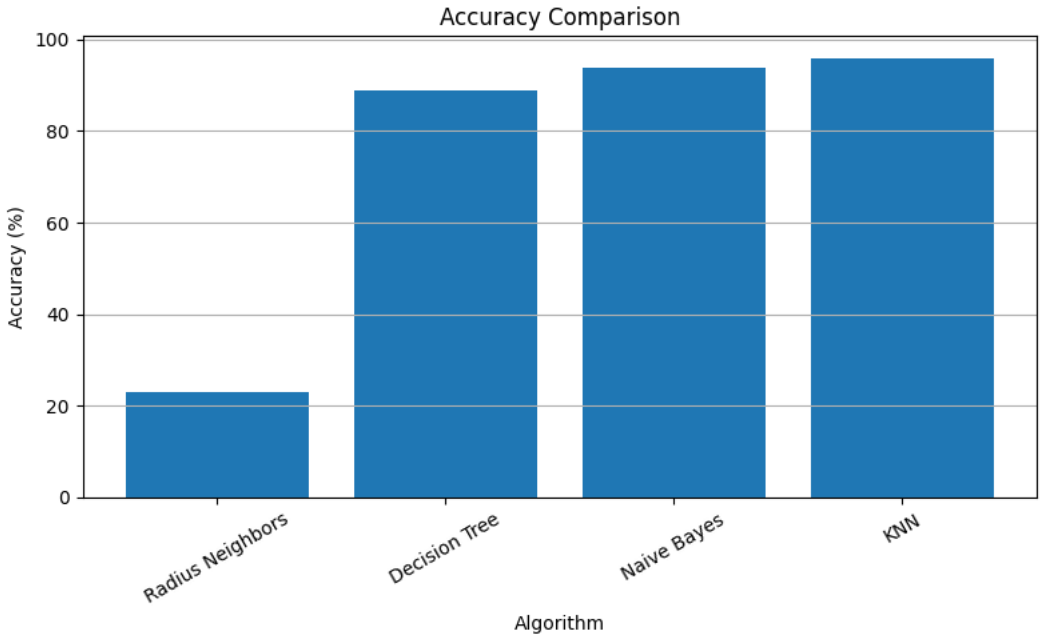


Fig. 9.4: Detailed performance comparison of classification models.

CHAPTER 10
CONCLUSION AND FUTURE SCOPE

This project successfully presented a fine-grained topic categorization system for BBC news articles using supervised document classification models. By integrating robust NLP preprocessing techniques with semantic feature extraction using GloVe word embeddings, the system effectively transformed unstructured news text into meaningful numerical representations. Multiple machine learning algorithms—namely Radius Neighbors Classifier, Decision Tree Classifier, Gaussian Naïve Bayes, and the proposed K-Nearest Neighbors (KNN) classifier—were implemented and evaluated to analyze their performance in classifying news articles into predefined categories such as business, entertainment, politics, sports, and technology. Comprehensive exploratory data analysis (EDA) helped in understanding word distributions, document lengths, linguistic patterns, and contextual relationships within the dataset, ensuring better feature quality and informed model selection. Performance evaluation using accuracy, precision, recall, F1-score, confusion matrices, and AUC–ROC curves demonstrated that the KNN classifier achieved superior performance compared to the existing baseline models. The instance-based learning nature of KNN, combined with semantic similarity captured by GloVe embeddings, enabled accurate and consistent predictions for fine-grained topic classification. Overall, the system proved to be efficient, scalable, and reliable for automated news categorization, significantly reducing the need for manual labeling and improving content organization and retrieval.

Future Scope

Although the proposed system delivers promising results, there are several opportunities for further enhancement and expansion. In the future, more advanced word and sentence embedding techniques such as Word2Vec, FastText, Doc2Vec, or contextual embeddings like BERT, RoBERTa, and GPT-based models can be integrated to capture deeper contextual and semantic relationships within news articles. These models may further improve classification accuracy, especially for ambiguous or overlapping topics.Additionally, the system can be extended to support multilingual news classification, enabling categorization of articles from global news sources beyond English.

REFERENCES

[1] Brank J., Grobelnik M., Milic-Frayling N., Mladenic D., "Interaction of Feature Selection Methods and Linear Classification Models", Proc. of the 19th International Conference on Machine Learning, Australia, 2002.

[2] Han X., Zu G., Ohyama W., Wakabayashi T., Kimura F., Accuracy Improvement of Automatic Text Classification Based on Feature Transformation and Multi-classifier Combination, LNCS, Volume 3309, Jan 2004, pp. 463-468

[3] Balcerzak, Bartłomiej & Jaworski, Wojciech & Wierzbicki, Adam. (2014). Application of TextRank Algorithm for Credibility Assessment. 451-454. 10.1109/WIIAT.2014.70.

[4] Ikonomakis, Emmanouil & Kotsiantis, Sotiris & Tampakas, V.. (2005). Text Classification Using Machine Learning Techniques. WSEAS transactions on computers. 4. 966- 974.

[5] Netzer, O., Lemaire, A., & Herzenstein, M. (2019). When Words Sweat: Identifying Signals for Loan Default in the Text of Loan Applications. Journal of Marketing Research, 56(6), 960–980.

[6] Huang, S., Cai, N., Pacheco, P. P., Narrandes, S., Wang, Y., & Xu, W. (2018). Applications of Support Vector Machine (SVM) Learning in Cancer Genomics. Cancer genomics & proteomics, 15(1), 41–51. https://doi.org/10.21873/cgp.20063

[7] Mihalcea, Rada. "Language independent extractive summarization." Proceedings of the ACL 2005 on Interactive poster and demonstration sessions. Association for Computational Linguistics, 2005.

[8] Huang, Hongzhao, et al. "Tweet Ranking Based on Heterogeneous

[9] Networks." COLING. 2012.

[10] Zu Eissen, S.M.; Stein, B. Genre classification of web pages. In Springer: Berlin/Heidelberg, Germany, 2004; pp. 256–269.

[11] Vidulin, V.; Luštrek, M.; Gams, M. Using genres to improve search engines. In Proceedings of the 1st International Workshop: Towards Genre-Enabled Search Engines: The Impact of Natural Language Processing, Borovets, Bulgaria, 30 September 2007; pp. 45–51.

[12] Daud S, Ullah M, Rehman A, Saba T, Damaševičius R, Sattar A. Topic Classification of Online News Articles Using Optimized Machine Learning Models. 2023; 12(1):16. https://doi.org/10.3390/computers12010016

[13] Rafat Y, Narayana P, Mohana RM, Srilatha K. LSTM-Based News Article Category Classification. 2025; 12(1):8. https://doi.org/10.3390/cmsf2025012008

[14] Piasta K, Kotas R. Comparative Analysis of Natural Language Processing Techniques in the Classification of Press Articles. 2025; 15(17):9559. https://doi.org/10.3390/app15179559

[15] Masri Y, Wang Z, Srirenganathan Malarvizhi A, Ahmed S, Stover T, Wong DWS, Jiang Y, Li Y, Liu Q, Bere M, et al. Comparative Analysis of BERT and GPT for Classifying Crisis News with Sudan Conflict as an Example. 2025; 18(7):420. https://doi.org/10.3390/a18070420

[16] Kalogeropoulos N-R, Ioannou D, Stathopoulos D, Makris C. On Embedding Implementations in Text Ranking and Classification Employing Graphs. 2024; 13(10):1897. https://doi.org/10.3390/electronics13101897

[17] Babić K, Martinčić-Ipšić S, Meštrović A. Survey of Neural Text Representation Models. 2020; 11(11):511. https://doi.org/10.3390/info11110511

[18] Harris S, Hadi HJ, Ahmad N, Alshara MA. Fake News Detection Revisited: An Extensive Review of Theoretical Frameworks, Dataset Assessments, Model Constraints, and Forward-Looking Research Agendas. 2024; 12(11):222. https://doi.org/10.3390/technologies12110222

[19] Petukhova A, Fachada N. MN-DS: A Multilabeled News Dataset for News Articles Hierarchical Classification. 2023; 8(5):74. https://doi.org/10.3390/data8050074

[20] Yuan L, Jiang H, Shen H, Shi L, Cheng N. Sustainable Development of Information Dissemination: A Review of Current Fake News Detection Research and Practice. 2023; 11(9):458. https://doi.org/10.3390/systems11090458

[21] Misra, R. Research& Services. Available online.

[22] D212digital. A Comprehensive Guide to Feature Engineering with N-Grams For Natural Language Processing. 2023. Available online.

[23] Manoj, V.; Devi, T. An Effective Approach for Newspaper Article Classification Using Multi-Class Support Vector Machine in Comparison with Binary Classifier to Improve Accuracy. In Proceedings of the 8th IEEE International Conference on Science, Technology, Engineering and Mathematics, ICONSTEM 2023, Chennai, India, 6–7 April 2023.

[24] Nigam, K. Comparing Classification Models. 2023. Available online.

[25] Foroughi, O. Multiclass_Text_Classification. 2021. Available online.

[26] Duan, D. News-Article-Classification. Available online.

[27] Barbaresi, A. Trafilatura: A Web Scraping Library and Command-Line Tool for Text Discovery and Extraction. In Proceedings of the ACL-IJCNLP 2021—59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, Proceedings of the System Demonstrations, Online, 1–6 August 2021; pp. 122–131.

[28] Germec, M. Text Preprocessing with Natural Language Processing (NLP). 2023. Available online.

[29] Silva, M.D. Preprocessing Steps for Natural Language Processing (NLP): A Beginner's Guide. 2023. Available online.

[30] Sarica, S.; Luo, J. Stopwords in Technical Language Processing. PLoS ONE 2021, 16, e0315195.

[31] Praveen Gujjar, J.; Prasanna Kumar, H.R.; Guru Prasad, M.S. Advanced NLP Framework for Text Processing. In Proceedings of the 2023 6th International Conference on Information Systems and Computer Networks, ISCON 2023, Mathura, India, 3–4 March 2023.

[32] Aditya, M.A.; Helen, A.; Suryana, I. Naïve Bayes and Maximum Entropy Comparison for Translated Novel's Genre Classification. J. Phys. Conf. Ser. 2021, 1722, 012007.

[33] Mazuelas, S.; Shen, Y.; Perez, A. Generalized Maximum Entropy for Supervised Classification. IEEE Trans. Inf. Theory 2022, 68, 2530–2550.

[34] Kruczek, J.; Kruczek, P.; Kuta, M. Are N-Gram Categories Helpful in Text Classification? In Computational Science—ICCS 2020; Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics); Springer: Cham, Switzerland, 2020; Volume 12138, pp. 524–537.

[35] Das, M.; Kamalanathan, S.; Alphonse, P. A Comparative Study on TF-IDF Feature Weighting Method and Its Analysis Using Unstructured Dataset. CEUR Workshop Proc. 2021, 2870, 98–107.

[36] Brownlee, J. A Gentle Introduction to the Bag-of-Words Model. 2019. Available online

[37] Gupta, A. Feature Selection Techniques in Machine Learning. 2023. Available online

[38] Morán-Fernández, L.; Bolón-Canedo, V. Finding a Needle in a Haystack: Insights on Feature Selection for Classification Tasks. J. Intell. Inf. Syst. 2024, 62, 459–483.

[39] Bayat, S.; Işık, G. Assessing the Efficacy of LSTM, Transformer, and RNN Architectures in Text Summarization. In Proceedings of the 5th International Conference on Applied Engineering and Natural Sciences, Konya, Turkey, 10–12 July 2023; Volume 1, pp. 813–820.

[40] Merritt, R. What Is a Transformer Model? 2022. Available online

[41] Bengi, İ.C. A.I. Transformers—The Beginner's Guide. 2023. Available online

[42] Khay, A. 5 Top Most Powerful Transformer Models 2023. 2023. Available online.

[43] Rao V, Sachdev J. A machine learning approach to classify news articles based on location. In Proceedings of the International Conference on Intelligent Sustainable Systems (ICISS), pp 863-867. https://doi.org/10.1109/iss1.2017.8389300Sun, C.; Qiu, X.; Xu, Y.; Huang, X. How to Fine-Tune BERT for Text Classification? In Chinese Computational Linguistics; Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics); Springer: Cham, Switzerland, Volume 11856, pp. 194–206.

[44] V, M.; T, D.; Kalaiyarasi, M. Classification of Newspaper Article Classification by Employing Support Vector Machine in Comparison with Perceptron to Improve Accuracy. In Proceedings of the Eighth International Conference on Science Technology Engineering and Mathematics (ICONSTEM), Chennai, India, 6–7 April 2023; p. 16. Beltagy, I.; Peters, M.E.; Cohan, A. Longformer: The Long-Document Transformer. arXiv 2020, arXiv:2004.05150.

[45] Jia, Y.; Chen, Z.; Yu, S. Reader emotion classification of news headlines. In Proceedings of the 5th International Conference on Smart Systems and Inventive Technology (ICSSIT), Tirunelveli, India, 23–25 January 2023; p. 16. [Google Scholar]Ott, M.; Edunov, S.; Grangier, D.; Auli, M. Scaling Neural Machine Translation. In Proceedings of the WMT 3rd Conference on Machine Translation, Brussels, Belgium.

[46] Beltagy, I.; Peters, M.E.; Cohan, A. Longformer: The Long-Document Transformer.

[47] Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, Ł.; Polosukhin, I. Attention Is All You Need. Adv. Neural Inf. Process. Syst.

[48] Ott, M.; Edunov, S.; Grangier, D.; Auli, M. Scaling Neural Machine Translation. In Proceedings of the WMT 2018—3rd Conference on Machine Translation, Brussels, Belgium.

[49] Popel, M.; Bojar, O. Training Tips for the Transformer Model. Prague Bull. Math. Linguist.

[50] Keskar, N.S.; Nocedal, J.; Tang, P.T.P.; Mudigere, D.; Smelyanskiy, M. On LargeBatch Training for Deep Learning: Generalization Gap and Sharp Minima. In Proceedings of the 5th International Conference on Learning Representations, ICLR Conference Track Proceedings, Toulon, France.

[51] Rish, I. An Empirical Study of the Naive Bayes Classifier. In Proceedings of the IJCAI Workshop on Empirical Methods in Artificial Intelligence, Seattle, WA, USA.

[52] Ray, S. Naive Bayes Classifier Explained: Applications and Practice Problems of Naive Bayes Classifier. 2024. Available online.

[53] Berger, A.L.; Della Pietra, V.J.; Della Pietra, S.A. A Maximum Entropy Approach to Natural Language Processing. Comput. Linguist.

[54] Maroulis, G. Comparison Between Maximum Entropy and Naïve Bayes Classifiers: Case Study; Appliance of Machine Learning Algorithms to an Odesk's Corporation Dataset. Ph.D. Thesis, Edinburgh Napier University, Edinburgh, UK.

[55] Ganitkevitch, J.; van Durme, B.; Callison-Burch, C. PPDB: The Paraphrase Database. In Proceedings of the 2nd Workshop on Computational Linguistics for Literature, CLfL at the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT, Atlanta, GA, USA, 9–15 June 2013; pp. 758–764.

[56] Manning, C.; Raghavan, P.; Schütze, H. Introduction to Information Retrieval; An Introduction to Information Retrieval; Cambridge University Press: Cambridge, UK.

[57] Karpiuk, M. Classification of Text Documents Based on Generalized Sequence Patterns. Ph.D. Thesis, Politechnika Warszawska, Wydział Elektroniki i Technik Informacyjnych Instytut Informatyki, Warsaw, Poland.

[58] Mohammed Badr, B.; Sameen Fatima, S. Using Skipgrams, Bigrams, and Part of Speech Features for Sentiment Classification of Twitter Messages. In Proceedings

of the 12th International Conference on Natural Language Processing, Trivandrum, India, 11–14 December 2015; pp. 268–275.

[59] Robinson, T. Bigram vs. Trigram. 1997. Available online (accessed on 1 December 2024).

[60] Berger, A.L.; Della Pietra, V.J.; Della Pietra, S.A. A Maximum Entropy Approach to Natural Language Processing. Comput. Linguis.

[61] Pouli, V.; Kafetzoglou, S.; Tsiropoulou, E.E.; Dimitriou, A.; Papavassiliou, S. Personalized multimedia content retrieval through relevance feedback techniques for enhanced user experience. In Proceedings of the 2015 13th International Conference on Telecommunications (ConTEL), Graz, Austria, 13–15 July 2015.

[62] Thai, M.T.; Wu, W.; Xiong, H. *Big Data in Complex and Social Networks*, 1st ed.; Chapman & Hall/CRC: Boca Raton, FL, USA, 2016.

[63] Li, H.; Yamanishi, K. Text classification using ESC-based stochastic decision lists. *Inf. Process. Manag.* **2002**, *38*, 343–361.

[64] Kadhim, A.I. Survey on supervised machine learning techniques for automatic text classification. *Artif. Intell. Rev.* **2019**, *52*, 273–292.

[65] Ko, Y.; Seo, J. Automatic text categorization by unsupervised learning. In Proceedings of the 18th conference on Computational linguistics—Volume 1, Saarbrücken, Germany, 31 July–4 August 2000; Volume 1, pp. 453–459.

[66] Kowsari, K.; Meimandi, K.J.; Heidarysafa, M.; Mendu, S.; Barnes, L.E.; Brown, D.E. Text Classification Algorithms: A Survey. *Information* **2019**, *10*, 150.

[67] Pedregosa, F.; Varoquaux, G.; Gramfort, A.; Michel, V.; Thirion, B.; Grisel, O.; Blondel, M.; Prettenhofer, P.; Weiss, R.; Dubourg, V.; et al. Scikit-learn: Machine Learning in Python. *J. Mach. Learn. Res.* **2011**, *12*, 2825–2830.

[68] Maas, A.L.; Daly, R.E.; Pham, P.T.; Huang, D.; Ng, A.Y.; Potts, C. Learning Word Vectors for Sentiment Analysis. In Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies, Portland, OR, USA, 19–24 June 2011; pp. 142–150.

[69] Pradhan, L.; Taneja, N.A.; Dixit, C.; Suhag, M. Comparison of Text Classifiers on News Articles. *Int. Res. J. Eng. Technol. (IRJET)* **2017**, *4*, 2513–2517.

[70] Rennie, J.D.M.; Rifkin, R. *Improving Multiclass Text Classification with the Support Vector Machine*; Technical Report; MIT Aritificial Intelligence Laboratory: Cambridge, MA, USA, 2001.

[71] Ghosh, M.; Sanyal, G. Performance Assessment of Multiple Classifiers Based on Ensemble Feature Selection Scheme for Sentiment Analysis. *Appl. Comput. Intell. Soft Comput.* **2018**, *2018*.

[72] Gamal, D.; Alfonse, M.; El-Horbarty, E.; M.Salem, A. Analysis of Machine Learning Algorithms for Opinion Mining in Different Domains. *Mach. Learn. Knowl. Extr.* **2019**, *1*, 224–234.

[73] Joachims, T. Text Categorization with Support Vector Machines: Learning with Many Relevant Features. In Proceedings of the European Conference on Machine Learning, Chemnitz, Germany, 21–23 April 1998.

[74] Do, T.N.; Lenca, P.; Lallich, S.; Pham, N.K. Classifying Very-High-Dimensional Data with Random Forests of Oblique Decision Trees. In *Advances in Knowledge Discovery and Management*; Springer: Berlin/Heidelberg, Germany, 2010; pp. 39–55.

[75] Aggarwal, C.C. *Machine Learning for Text*; Springer: Berlin/Heidelberg, Germany, 2018.

[76] Salazar, D.A.; Velez, J.I.; Salazar, J.C. Comparison between SVM and Logistic Regression: Which One is Better to Discriminate? *Rev. Colomb. Estadística* **2012**, *35*, 223–237.

[77] Sperandei, S. Understanding logistic regression analysis. *Biochem. Med.* **2014**, *24*, 12–18.

[78] Yue, S.; Li, P.; Hao, P. SVM classification: Its contents and challenges. *Appl. Math. J. Chin. Univ.* **2003**, *18*, 332–342.

[79] Kotsiantis, S.B. Decision trees: A recent overview. *Artif. Intell. Rev.* **2013**, *39*, 261–283.

[80] Kamiński, B.; Jakubczyk, M.; Szufel, P. A framework for sensitivity analysis of decision trees. *Cent. Eur. J. Oper. Res.* **2017**, *26*, 135–159.